影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 11 章 小波與正交轉換

本章學習目標

- 說明正交轉換的共同思想，理解「換一組基底表示影像」為何有利於壓縮與分析。
- 闡述傅立葉轉換「有頻率、無位置」的根本弱點，說明短時傅立葉與小波如何回應。
- 推導 Haar 小波與多解析度分析，實作二維離散小波轉換的多層分解。
- 說明離散餘弦轉換(DCT)的原理，完整拆解 JPEG 壓縮的五個步驟與方塊效應成因。
- 比較 JPEG 與 JPEG 2000(小波)的差異，解釋小波為何避免方塊效應。
- 運用小波於影像去雜訊、融合與浮水印，並認識其在深度學習時代的定位。

叫獸開講|FBI 的指紋危機：三千五百萬張卡片與小波的救援

1990 年代初，美國聯邦調查局(FBI)面臨一個幾乎要把系統壓垮的難題。它的檔案庫裡堆放著超過三千五百萬張紙本指紋卡，還有大量待分類的積壓，而每天湧入的指紋比對請求又不斷增加。唯一的出路是全面數位化——但這裡藏著一個駭人的數字：以當年的技術把整個指紋庫掃描存檔，將產生大約一千一百四十兆位元組(超過一千 TB)的影像資料。光是儲存就是天文數字，更別提用當年的數據機傳輸一張指紋卡要花上好幾分鐘。不壓縮，數位化根本不可能。

最直覺的選擇是用現成的 JPEG 壓縮。但 FBI 與國家標準暨技術研究院(NIST)的早期測試發現一個致命問題：JPEG 在高壓縮率下會產生「方塊效應」——影像被切成 8×8 的小方塊各自處理，壓得愈狠，方塊之間的接縫就愈明顯，像一面貼

歪的磁磚牆。對一般照片這或許可以忍受，但對指紋是災難：指紋辨識靠的是脊線的細微特徵點(minutiae)——分岔、端點、間距，而 JPEG 的方塊效應與高頻細節流失，恰恰會破壞或偽造這些用於法律鑑識的關鍵特徵。一個被壓縮演算法「抹掉」或「無中生有」的特徵點，在法庭上可能就是天壤之別。

FBI 找上了洛斯阿拉莫斯國家實驗室。由 FBI 的 Tom Hopper 與洛斯阿拉莫斯的 Jonathan Bradley、Chris Brislawn 領軍的團隊，最終採用了一套基於小波的方案，稱為「小波純量量化」(Wavelet Scalar Quantization,WSQ)，並在 1993 年定為國家標準。與 JPEG 把影像硬切成方塊不同，小波轉換是對整張影像做多解析度分解，不存在方塊邊界，因此從根本上避免了方塊效應。WSQ 能在 20:1 的壓縮率下，達到人眼幾乎無法察覺的品質損失，同時保留鑑識級的脊線細節。三千五百萬張指紋卡的數位化危機，就這樣被一個當時還很年輕的數學工具——小波——化解了。

這個故事點出了本章的核心。第 6 章我們讚嘆傅立葉轉換的威力，但也埋下一個伏筆：它是「全域」的，只告訴你影像含有哪些頻率，卻不告訴你這些頻率出現在哪裡。JPEG 用「切成小方塊」來勉強取得局部性，代價就是方塊效應。而小波提供了一個更優雅的答案：它能同時告訴你「有哪些頻率」與「頻率在哪裡」——這就是多解析度分析。本章要講的，正是一場「如何同時掌握頻率與位置」的數學奮鬥史，以及它如何撐起你每天使用的影像壓縮。

思考題：JPEG 為了取得「局部性」而把影像切成 8×8 方塊，結果付出了方塊效應的代價。小波用不切塊的方式達到局部性。這讓你想到什麼工程上的通則?(提示：當一個系統為了某個好處而做出粗暴的妥協時，往往存在更精巧的替代方案)

11.1 正交轉換的共同思想

11.1.1 換一組基底看世界

本章與第 6 章共享一個深刻的思想：把影像從一組基底轉換到另一組基底表示。

第 6 章的傅立葉轉換，是把影像表示成一堆「不同頻率的弦波(斜條紋圖)」的加權和;本章的離散餘弦轉換與小波轉換，只是換用不同的基底函數，但精神完全一致——都是在問「這張影像，用這組積木來拼，各需要多少?」

為什麼要換基底?因為在一個好的基底下，影像的資訊會變得「稀疏」——絕大部分的能量集中在少數幾個係數上，其餘係數接近於零。而稀疏正是壓縮的黃金鑰匙：只要記住那少數重要的係數、把大量接近零的係數丟棄或粗略記錄，就能用極少的資料重建出接近原貌的影像。這是所有轉換式壓縮(JPEG、JPEG 2000)的共同原理。自然影像在空間域(像素域)並不稀疏(每個像素都攜帶資訊)，但轉換到頻率域或小波域後就變得非常稀疏——這就是轉換的價值。

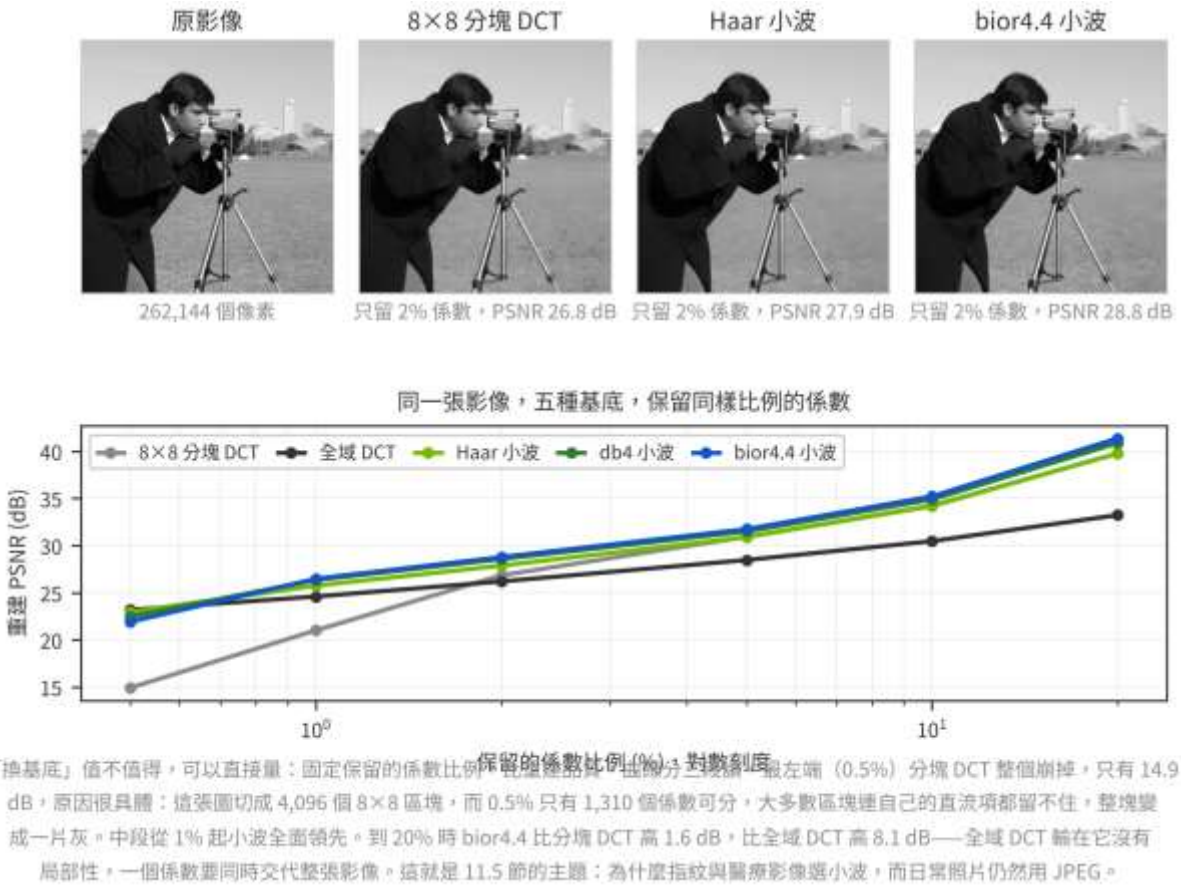
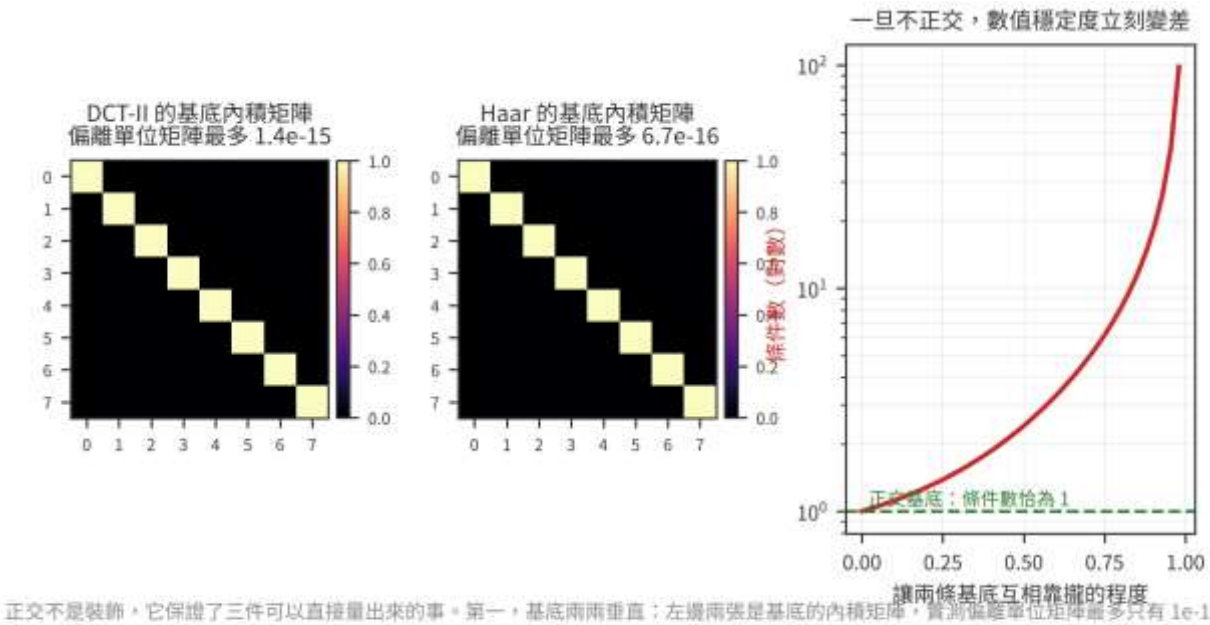


圖 11-1　三種基底看同一張影像：稀疏程度決定壓縮的本錢

11.1.2 正交的意義

這些轉換之所以稱為「正交」(Orthogonal)，是因為它們的基底函數彼此正交—

—用向量的語言說，任兩個基底函數的內積為零，彼此獨立、互不重疊。正交性帶來三個好處：轉換可逆(能無失真地變回原影像)、能量守恆(轉換前後總能量不變，即 Parseval 定理，這讓我們能在轉換域中判斷丟棄某係數會損失多少能量)、以及係數之間互不干擾(改動一個係數不影響其他係數的意義)。第 6 章的傅立葉、本章的 DCT 與正交小波，都具備這些性質。



正交不是裝飾，它保證了三件可以直接量出來的事。第一，基底兩兩垂直：左邊兩張是基底的內積矩陣，實測偏離單位矩陣最多只有 1e-15（DCT）與 7e-16（Haar），也就是浮點誤差的量級。第二，能量守恆（Parseval 定理）：一段 4096 點的隨機訊號，時域能量 4088.2，DCT 域 4088.2，Haar 域 4088.2，三者相同——所以「丟掉小係數」造成的誤差可以直接由被丟掉的係數平方和算出來，不必重建回去再比對。第三，數值穩定：正交基底的條件數恰為 1，而右圖讓兩條基底逐漸靠攏，條件數就一路竄到 98;基底愈接近平行，係數求解與反轉換就愈不穩定，輸入的一點點誤差會被放大同樣的倍數（這正是 7.1.5 節條件數的老問題）。

圖 11-2　正交性的三個好處：可逆、能量守恆、係數互不干擾

11.2 傅立葉的弱點與小波的動機

11.2.1 有頻率，無位置

第 6.6.5 節已經預告過傅立葉轉換的根本弱點，這裡完整展開。傅立葉轉換用「無限延伸的弦波」作為基底——每一個弦波都從負無窮延伸到正無窮，在整張影像上均勻存在。這帶來一個後果：當你在頻譜上看到某個頻率成分很強，你知道「這張影像含有這個頻率」，但你完全不知道「這個頻率來自影像的哪個位置」。

用一個聽覺的比喻：傅立葉分析一首樂曲，能告訴你整首曲子用到了哪些音符，

卻無法告訴你這些音符出現的先後順序——它把樂譜壓扁成了一張「音符清單」。對於平穩(statistics 不隨位置改變)的訊號，這不成問題;但影像恰恰是高度非平穩的：左半邊可能是平滑的天空(只有低頻)，右半邊可能是茂密的樹葉(充滿高頻)。傅立葉把這兩者的頻率混在一起報告，喪失了寶貴的位置資訊。

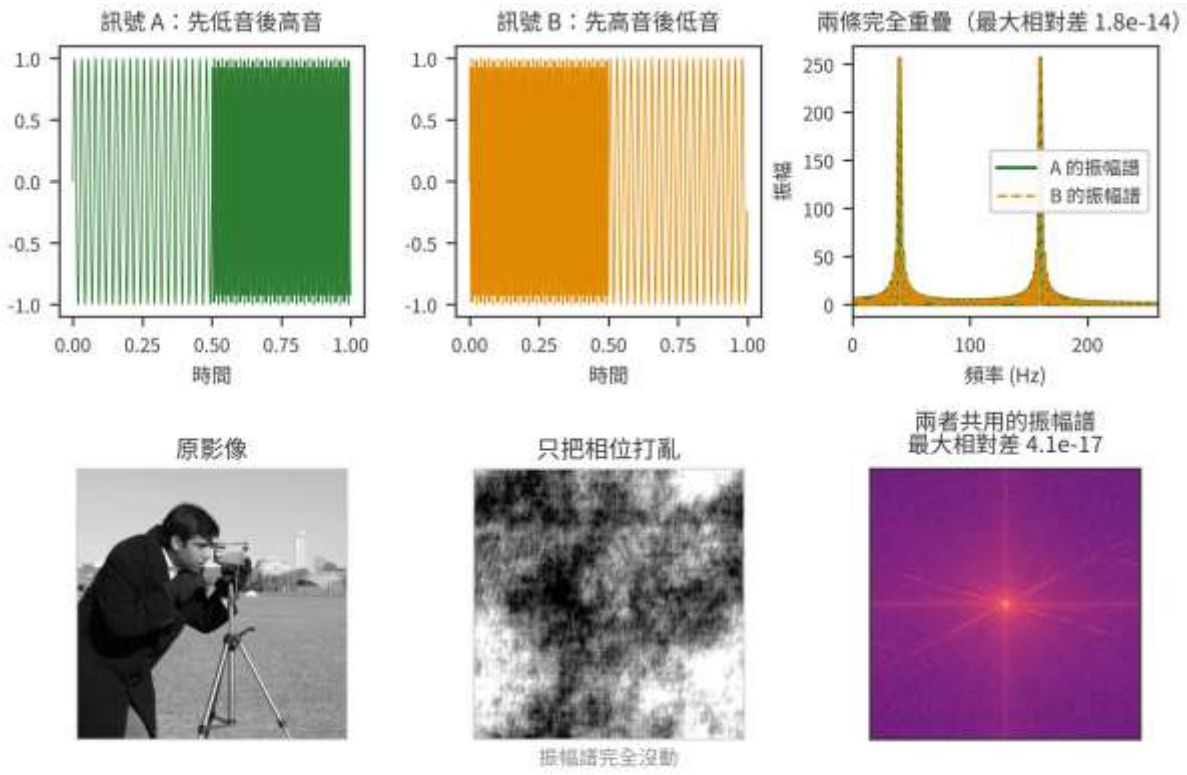


圖 11-3　有頻率、無位置：兩段訊號的頻譜完全相同

11.2.2 短時傅立葉與測不準

第一個補救是短時傅立葉轉換(STFT)：把影像切成小塊，對每一塊各自做傅立葉轉換，如此就能知道「某頻率出現在某塊」。JPEG 的 8×8 分塊 DCT 正是這個思路的體現。但 STFT 有一個先天的困境：窗口大小固定。窗口切得大，頻率分辨得準、但位置模糊;窗口切得小，位置定得準、但頻率模糊。你無法讓兩者同時精準——這就是訊號處理版的測不準原理(第 6.3.2 節提過「函數不可能在兩個域同時很窄」)。JPEG 的方塊效應，本質上就是固定窗口大小付出的代價。

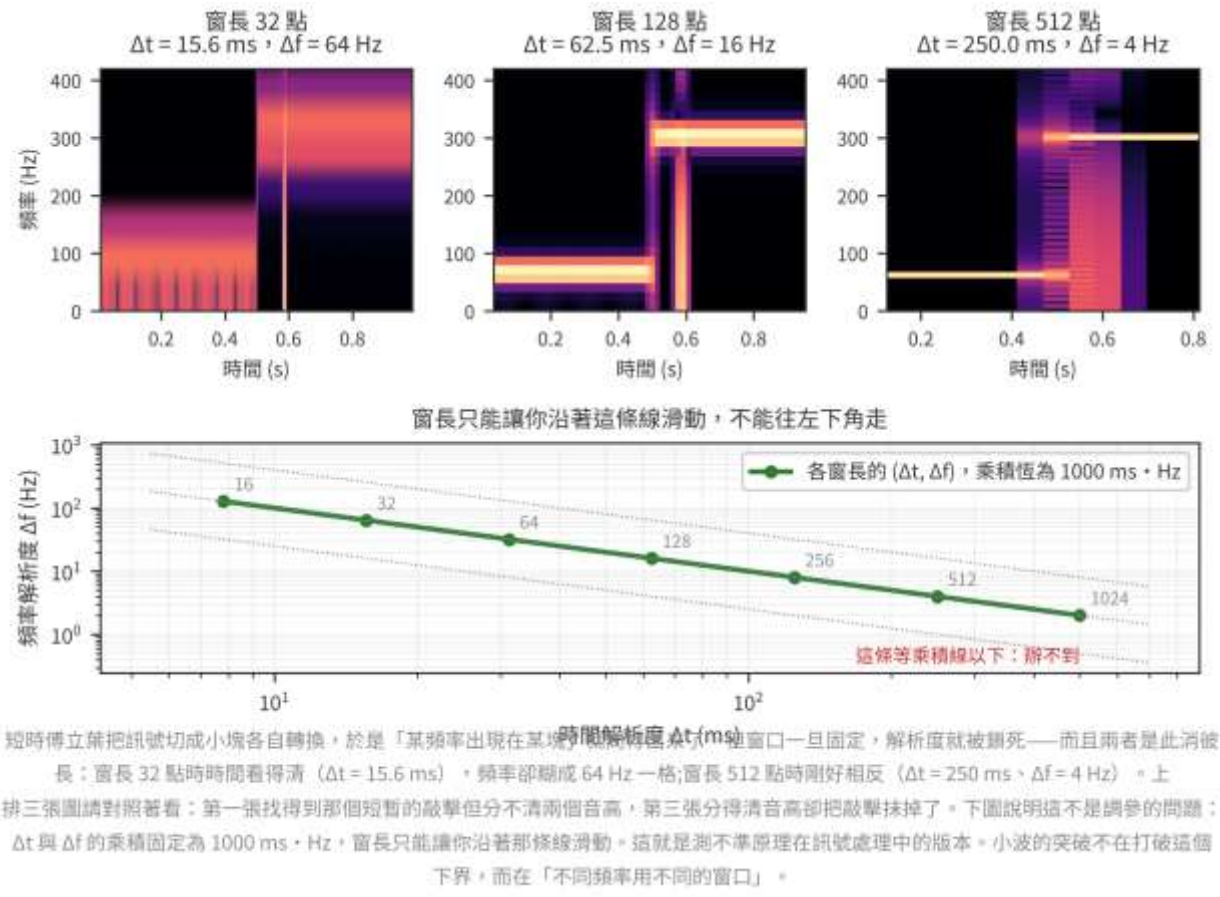


圖 11-4　測不準：窗口愈短，位置愈準而頻率愈糊

11.2.3 小波的洞見：讓窗口大小自己調整

小波的天才之處，在於它放棄了「固定窗口」。它的觀察是：低頻成分變化緩慢，需要長窗口才能看清(犧牲位置換頻率精度);高頻成分變化劇烈且短暫，需要短窗口才能定位(犧牲頻率換位置精度)。小波用一個「有限長度的小波形」(mother wavelet)作為基底，透過縮放(scaling，拉長或壓縮波形以對應不同頻率)與平移(translation，移動波形以對應不同位置)，自動實現「低頻用長窗、高頻用短窗」的自適應分析。

這正是多解析度分析(Multi-Resolution Analysis)的精髓：在粗的尺度上看大輪廓(低頻、低空間解析度)，在細的尺度上看小細節(高頻、高空間解析度)，不同尺度各司其職。它同時提供了頻率與位置資訊，徹底回應了傅立葉的弱點——這也是小波能避免方塊效應的根本原因：它從一開始就不需要把影像切成方塊。

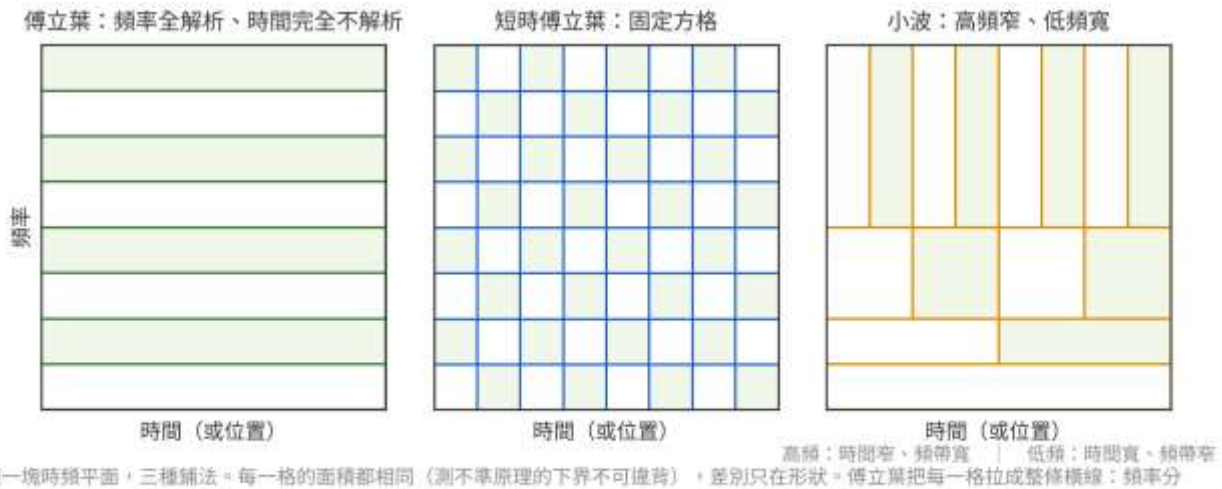


圖 11-5　時頻平面的三種鋪法：傅立葉、短時傅立葉、小波

11.3 Haar 小波與多解析度分析

11.3.1 最簡單的小波：Haar

Haar 小波是最早、也最簡單的小波，早在 1909 年就由 Alfréd Haar 提出(比小波理論的正式成熟早了數十年)。它的一維運算簡單到可以手算：對相鄰的一對數值，計算它們的平均(得到「概略」資訊，對應低頻)與差值的一半(得到「細節」資訊，對應高頻)。例如數列 [8, 4]：平均是 6、差的一半是 2，於是 [8, 4] 被表示成 [6, 2]——其中 6 是概略、2 是細節。要還原也容易：8 = 6 + 2、4 = 6 - 2。這個「平均與差分」的配對，就是最基本的小波分解。

對整個數列反覆施行：先兩兩配對得到一半長度的概略序列與一半長度的細節序列，再對概略序列繼續兩兩配對……如此層層遞進，就是多解析度分解。每往上一層，概略的解析度減半(看得更粗略)，而每一層的細節被分別保留。這解釋了小波「多解析度」之名的由來，也說明了它與傅立葉的根本不同：傅立葉一次把訊號攤平成頻譜，小波則是一層一層地剝出不同尺度的細節。

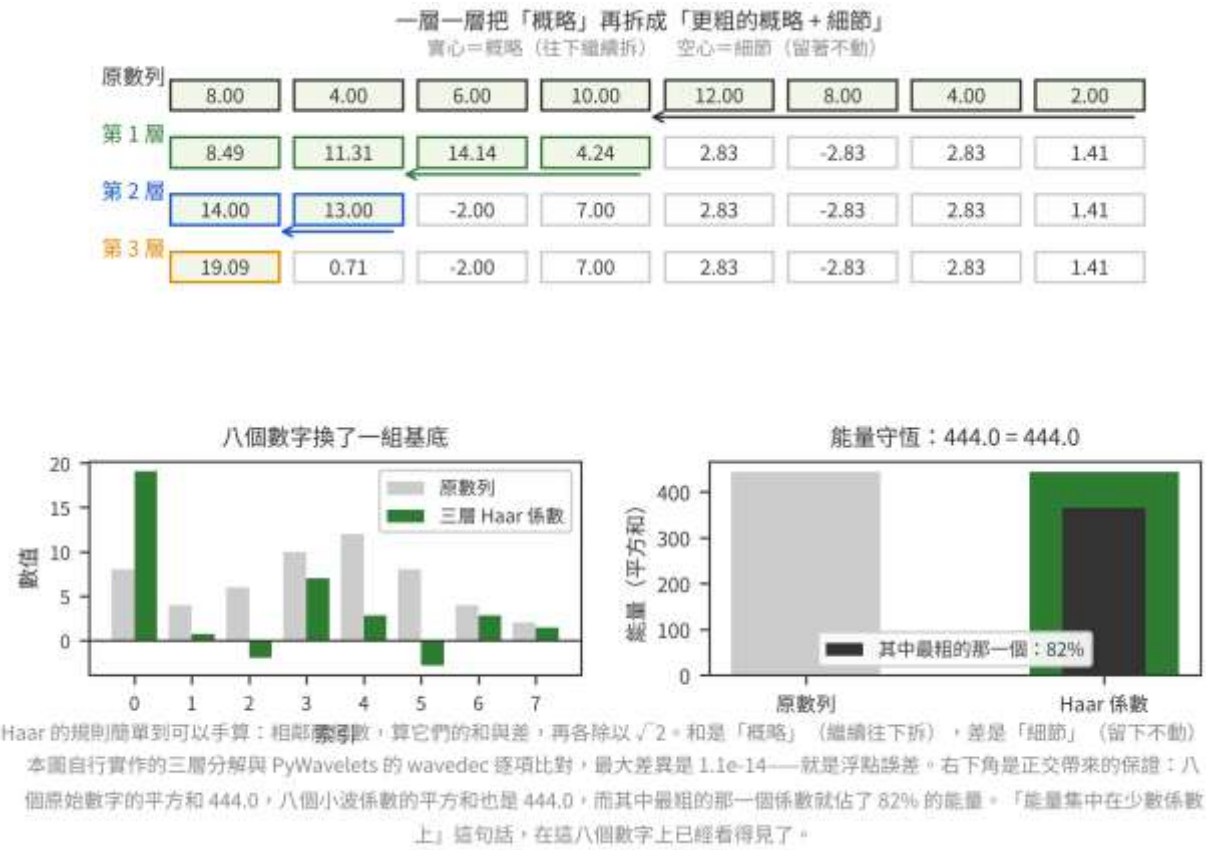


圖 11-6　Haar 小波的一維手算：平均與差分，層層遞進

11.3.2 二維小波轉換與四個子帶

把一維小波推廣到二維影像的標準做法是：先對每一列做一維小波轉換(水平方向)，再對每一行做一維小波轉換(垂直方向)。一層轉換後，影像被分解成四個子帶(subband)，各佔原影像四分之一大小：

1. LL(低-低)子帶：水平垂直都取概略，是原影像的縮小版(縮略圖)，集中了絕大部分能量。
2. LH(低-高)子帶：水平概略、垂直細節，凸顯水平方向的邊緣。
3. HL(高-低)子帶：水平細節、垂直概略，凸顯垂直方向的邊緣。
4. HH(高-高)子帶：兩方向都取細節，凸顯對角線方向的細節與雜訊。

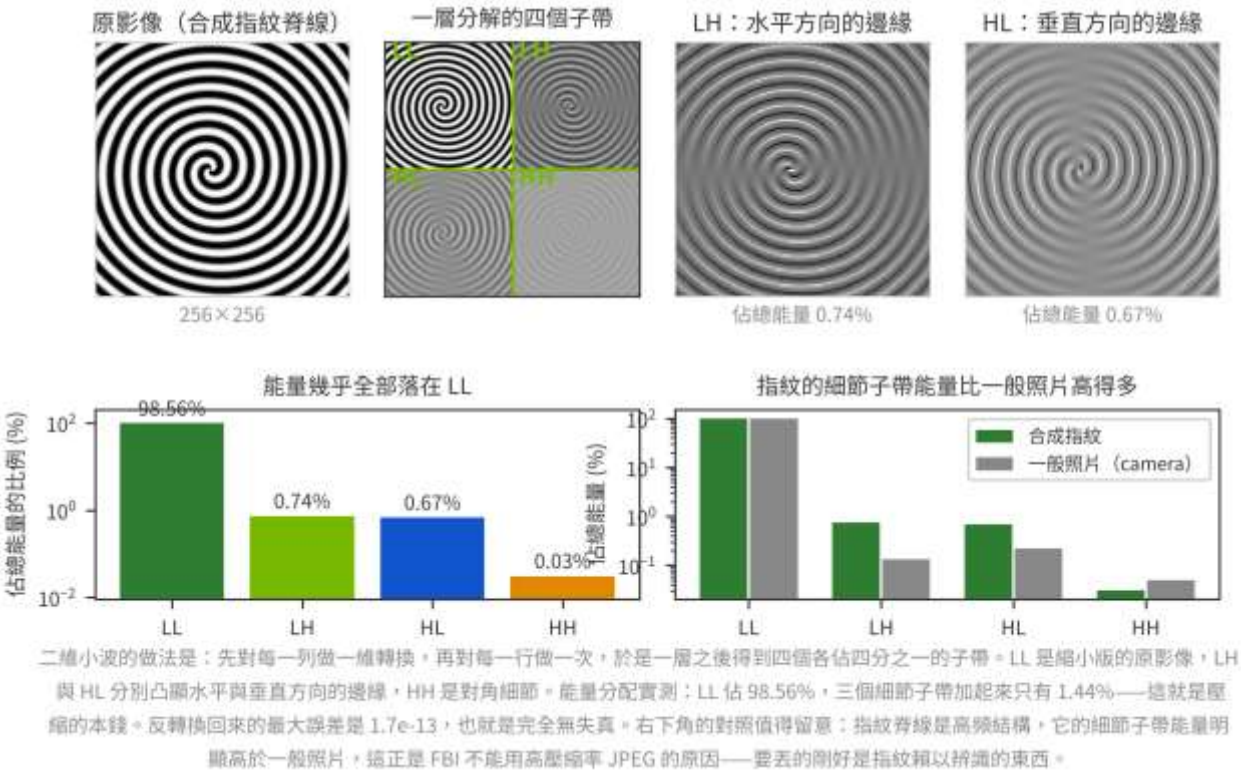


圖 11-7　二維一層分解的四個子帶

多層分解時，只對 LL 子帶(縮略圖)繼續往下分解，形成一個由粗到細的金字塔結構。這個結構的美妙之處在於：大部分能量集中在最頂層的小小 LL 子帶，而其他子帶的係數大多接近於零(自然影像的細節是稀疏的)——這正是 11.1.1 節說的「稀疏性」，壓縮的機會就藏在這裡。

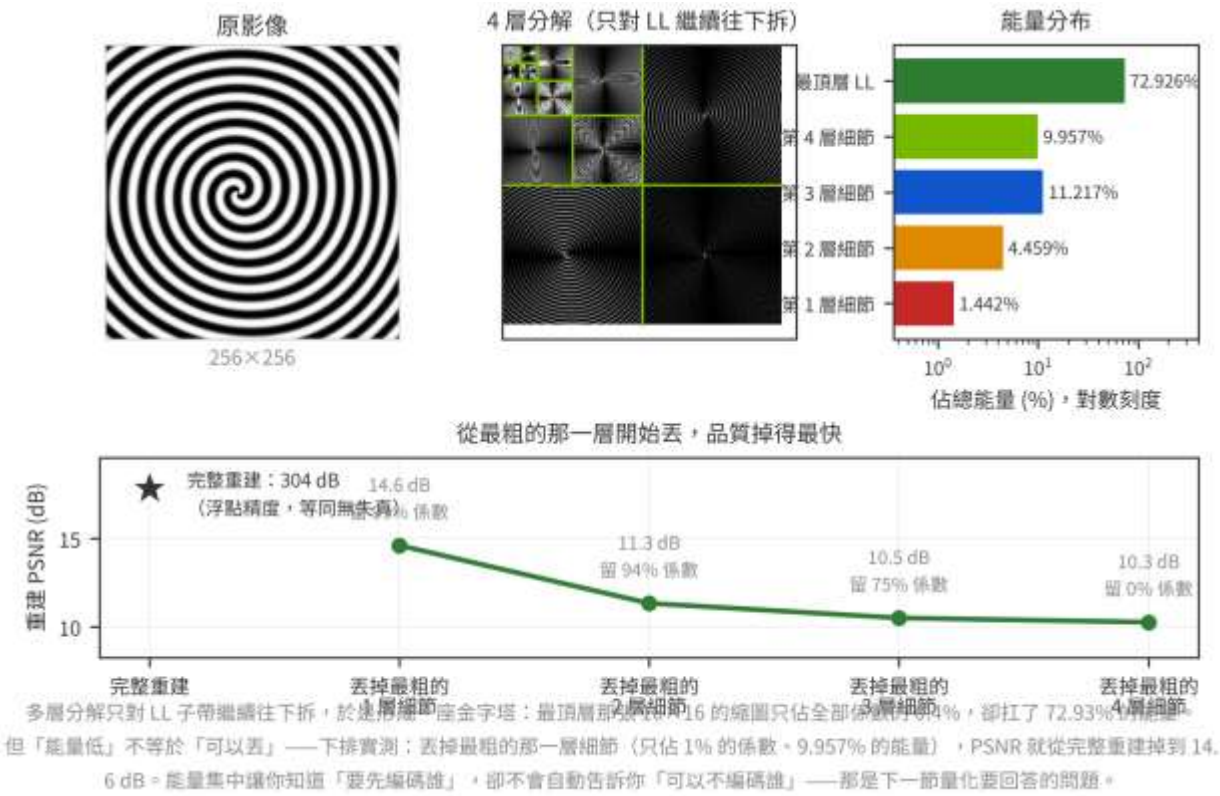


圖 11-8　多層分解的金字塔：只對 LL 續分，係數總數不變

程式 11-1 安裝 PyWavelets

```
# 安裝小波處理套件
pip install PyWavelets
```

程式 11-2 二維小波分解與重建

```
import cv2, numpy as np, pywt

gray = cv2.imread("fingerprint.jpg", cv2.IMREAD_GRAYSCALE).astype(float)

# --- 單層二維 Haar 小波轉換 ---
coeffs = pywt.dwt2(gray, "haar")
LL, (LH, HL, HH) = coeffs
print("四個子帶尺寸:", LL.shape)      # 各為原圖的四分之一

# 把四個子帶拼成一張圖觀察
def norm(x):
    return cv2.normalize(x, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
top = np.hstack([norm(LL), norm(LH)])
bot = np.hstack([norm(HL), norm(HH)])
cv2.imwrite("wavelet_subbands.png", np.vstack([top, bot]))

# --- 多層分解(三層) ---
coeffs_multi = pywt.wavedec2(gray, "haar", level=3)
arr, slices = pywt.coeffs_to_array(coeffs_multi)
cv2.imwrite("wavelet_pyramid.png", norm(arr))

# --- 反轉換:確認可完整還原 ---
```

```
recon = pywt.idwt2(coeffs, "haar")
print("還原最大誤差:", np.abs(gray - recon).max())    # 接近 0
```

除了 Haar，實務上常用的小波還有 Daubechies(db 系列，由 Ingrid Daubechies 於 1988 年提出，是小波理論成熟的里程碑)、Biorthogonal(雙正交，JPEG 2000 採用)等。不同的小波在「平滑度」與「支撐長度」上各有取捨，選擇適合訊號特性的小波，是小波應用的一門學問——FBI 的 WSQ 正是精心挑選了適合指紋脊線特性的小波。

11.3.3 推導：Haar 的 1/√2 為什麼不能省

很多教科書把 Haar 分解寫成「平均與差的一半」——概略 $a = (x_1+x_2)/2$、細節 $d = (x_1-x_2)/2$，本章 11.3.1 節也是這樣介紹的，因為它最好手算。但真正的 Haar 小波轉換用的是另一組係數：$a = (x_1+x_2)/\sqrt{2}$、$d = (x_1-x_2)/\sqrt{2}$。差一個 $\sqrt{2}$ 看似小事，其實決定了這個轉換算不算「正交」。

先驗算能量。取 $[x_1, x_2] = [8, 4]$。用 1/2 的版本：$a = 6$、$d = 2$，轉換後的能量是 $6^2 + 2^2 = 40$，而原始能量是 $8^2 + 4^2 = 80$——能量少了一半。用 $1/\sqrt{2}$ 的版本：$a = 12/\sqrt{2} \approx 8.485$、$d = 4/\sqrt{2} \approx 2.828$，能量是 $72 + 8 = 80$，與原始完全相同。

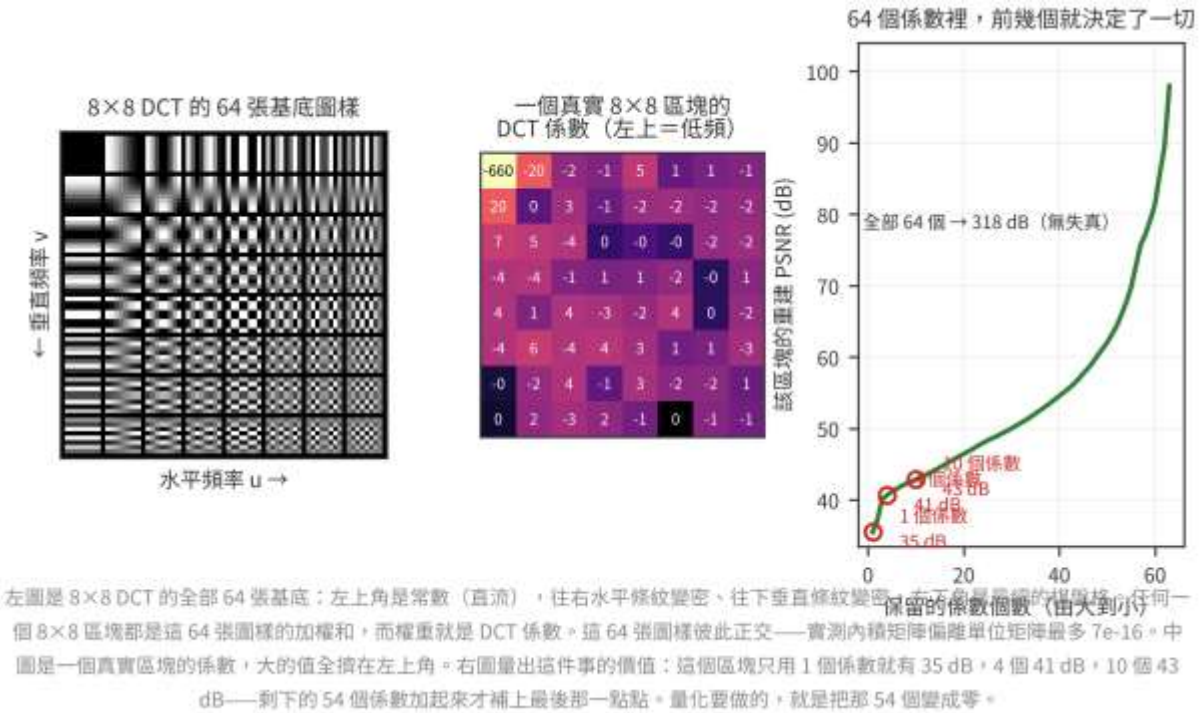
為什麼會這樣?把一層 Haar 寫成矩陣乘法 $y = H\,x$，其中 $H = c\cdot[[1, 1], [1, -1]]$，$c$ 是待定的常數。正交矩陣的定義是 $H^{\mathsf{T}}H = I$。計算 $H^{\mathsf{T}}H = c^2\cdot[[2, 0], [0, 2]] = 2c^2\cdot I$，要讓它等於單位矩陣，必須 $2c^2 = 1$，也就是 $c = 1/\sqrt{2}$。換句話說，$1/\sqrt{2}$ 不是美觀的選擇，而是「正交」這個條件唯一允許的值。

這件事為什麼重要?回到 11.1.2 節列的三個好處。正交保證了 Parseval 定理成立：轉換前後總能量相等。而壓縮的整套邏輯都建立在這條定理上——我們之所以敢說「丟掉小係數只損失一點點」，是因為係數的平方和就是影像的能量，丟掉一個大小為 ε 的係數，造成的重建誤差能量恰好也是 ε^2，不多也不少。若轉換不正交，係數的大小與它對重建的貢獻就脫鉤了：一個看起來很小的係數可能在反轉換時被放大成一片明顯的失真，「挑大的留下」這個策略也就失去了依據。

實務上 PyWavelets 的 pywt.dwt2(gray, "haar") 用的正是 1/√2 的正規化版本。所以如果你照著 11.3.1 節手算出 [6, 2]，再拿去和程式輸出比對，會發現程式給的是 [8.485, 2.828]——兩者差一個 √2 倍，都不是錯的，只是前者方便心算、後者保有正交性。要判斷手上的實作屬於哪一種，最快的方法就是算一次能量：若轉換前後的平方和相等，它就是正交版本。

11.4 離散餘弦轉換與 JPEG 壓縮

在小波稱霸之前(以及在絕大多數日常照片中至今)，影像壓縮的主力是離散餘弦轉換(DCT)與建立其上的 JPEG。理解 JPEG，是理解影像壓縮不可跳過的一課，它也把第 3、6、8 章的許多概念串在一起。



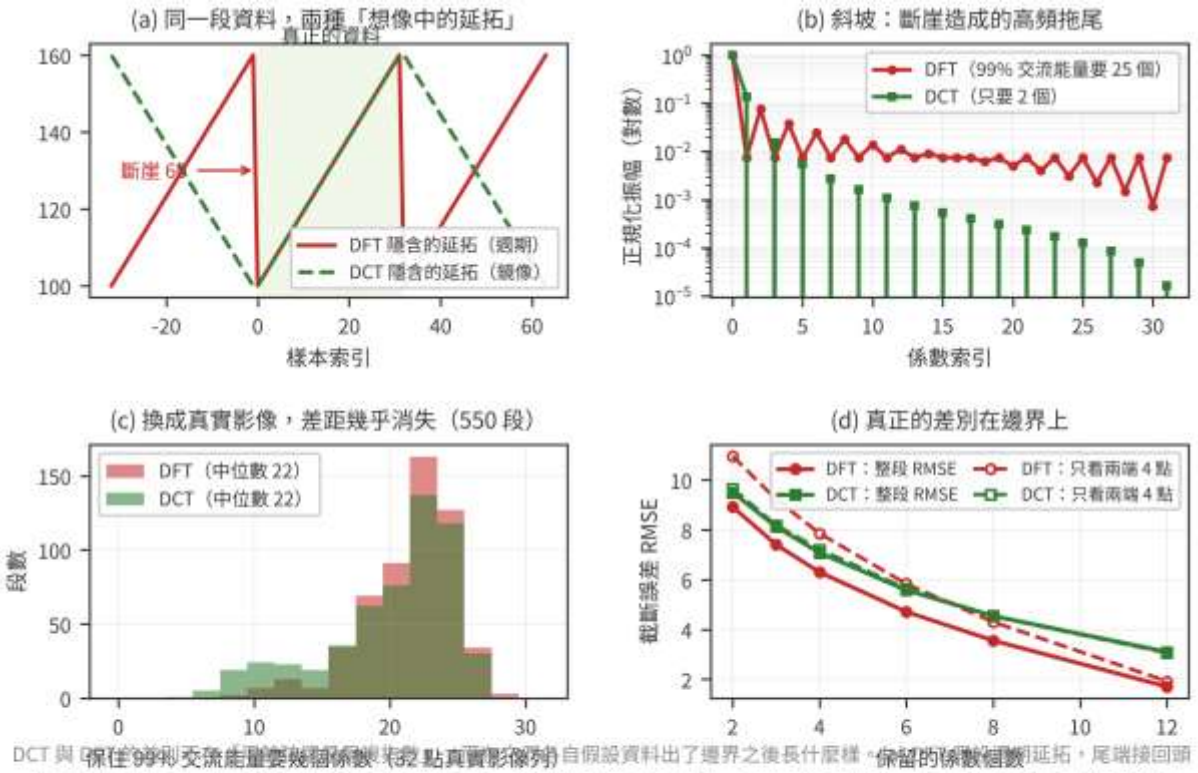
左圖是 8×8 DCT 的全部 64 張基底：左上角是常數（直流），往右水平條紋變密、往下垂直條紋變密，左下角是兩種條紋都最密。任何一個 8×8 區塊都是這 64 張圖樣的加權和，而權重就是 DCT 係數。這 64 張圖樣彼此正交——實測內積矩陣偏離單位矩陣最多 7e-16。中圖是一個真實區塊的係數，大的值全擠在左上角。右圖量出這件事的價值：這個區塊只用 1 個係數就有 35 dB，4 個 41 dB，10 個 43 dB——剩下的 54 個係數加起來才補上最後那一點點。量化要做的，就是把那 54 個變成零。

圖 11-9　DCT 的 64 個基底影像

11.4.1 離散餘弦轉換

DCT 是傅立葉家族的近親，差別在於它只使用餘弦函數作為基底。這帶來一個實用的好處：對實數影像做 DCT 得到的係數也是實數(不像傅立葉會產生複數)，處理起來更簡便。更重要的是，DCT 有優異的能量集中特性：對自然影像的區塊做 DCT，能

量會高度集中在少數幾個低頻係數(左上角)上，而高頻係數(右下角)大多接近零——這種稀疏性正是壓縮的基礎(呼應 11.1.1 節)。



DCT 與 [illegible] 自假設資料出了邊界之後長什麼樣。(a) 同一段資料，兩種「想像中的延拓」：DFT 假設訊號週期延拓，尾端接回頭端，中間出現一道高度 60 的斷崖;DCT 假設鏡像延拓，接縫完全連續（斷崖 0）。(b) 斷崖就是高頻：同一段斜坡 DFT 要 25 個係數才保得住 99% 的交流能量，DCT 只要 2 個。但 (c) 換成真實影像的 32 點列，兩者的中位數都是 22——課本常說的「DCT 比較會集中能量」，在自然影像上其實只是微幅領先。真正的差別在 (d)：只保留 2 個係數時，DFT 的整段誤差反而略低，但兩端 4 點的誤差是 11.0，DCT 只有 9.7。那道斷崖的振鈴全擠在邊界上，而分塊編碼恰恰是把一堆邊界拼在一起——這才是 JPEG 選 DCT 的實質理由（外加一個很現實的好處：DCT 的輸出是實數，不必做複數運算，也不必存虛部）。

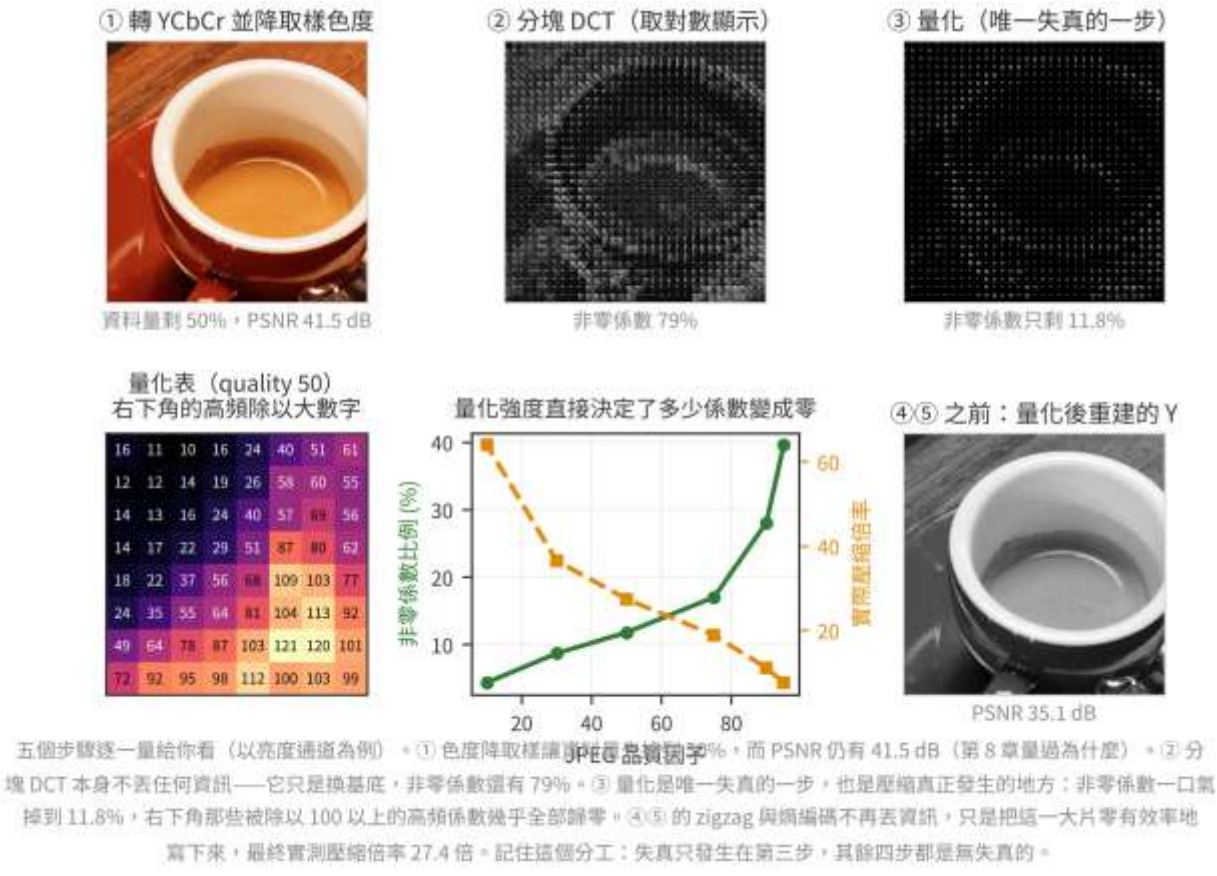
圖 11-10　DCT 比 DFT 更能集中能量?真正的差別在邊界

11.4.2 JPEG 壓縮的五個步驟

JPEG 是一條精心設計的管線，每個步驟都呼應本書前面的概念：

5. 色彩空間轉換與降取樣：先把 RGB 轉為 YCbCr(第 8 章)，分離亮度 Y 與色度 Cb、Cr;因為人眼對色度不敏感，對 Cb、Cr 做 4:2:0 降取樣(解析度砍四分之一)，這一步就先省下一半資料且幾乎無感——這是第 8 章「人眼對色度遲鈍」的直接應用。
6. 分塊與 DCT：把每個通道切成 8×8 的小區塊，對每一塊各自做 DCT。切塊正是 11.2.2 節短時傅立葉的思路——用分塊取得局部性，但也埋下方塊效應的種子。

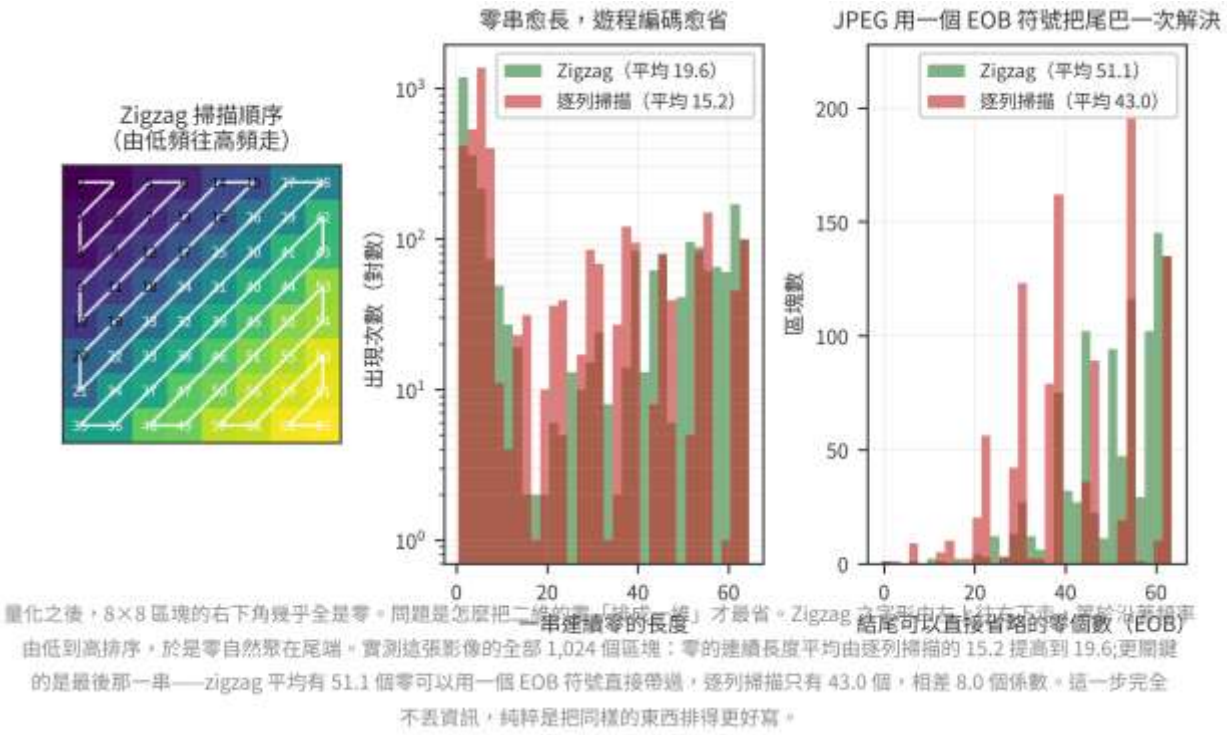
7. 量化(Quantization)：這是 JPEG 唯一「失真」的步驟，也是壓縮的核心。用一張「量化表」把每個 DCT 係數除以對應的量化步長再取整數——高頻係數用較大的步長(除以大數字)，許多因此變成零。品質參數(quality)控制的正是量化表的縮放：品質愈低、量化步長愈大、被歸零的高頻愈多、檔案愈小、失真愈大。這一步利用了人眼對高頻細節較不敏感的特性，大膽丟棄「看不太出來」的資訊。
8. Zigzag 掃描與遊程編碼：量化後，8×8 區塊的右下角(高頻)佈滿了零。以 zigzag(之字形)順序掃描，把二維區塊排成一維序列，能讓這些零連續排在一起，再用遊程編碼(把「連續 20 個零」記成一個符號)大幅壓縮。
9. 熵編碼(Entropy Coding)：最後用 Huffman 編碼(或算術編碼)對結果做無失真壓縮，把出現頻繁的符號用較短的碼表示。這一步不丟資訊，純粹榨乾統計冗餘。



五個步驟逐一量給你看（以亮度通道為例）。① 色度降取樣讓資料量剩 50%，而 PSNR 仍有 41.5 dB（第 8 章量過為什麼）。② 分塊 DCT 本身不丟任何資訊——它只是換基底，非零係數還有 79%。③ 量化是唯一失真的一步，也是壓縮真正發生的地方：非零係數一口氣掉到 11.8%，右下角那些被除以 100 以上的高頻係數幾乎全部歸零。④⑤ 的 zigzag 與熵編碼不再丟資訊，只是把這一大片零有效率地寫下來，最終實測壓縮倍率 27.4 倍。記住這個分工：失真只發生在第三步，其餘四步都是無失真的。

圖 11-11　JPEG 五個步驟的實際數據流

解壓縮則是完全反向的過程。整條管線中，只有第三步「量化」是不可逆的失真來源——這也是「反覆存檔 JPEG 會愈存愈糊」這個說法的由來。不過這句話需要補上條件：若每次都用同一個品質設定、而且影像沒有被裁切或移動，再量化其實近乎不動（因為係數已經落在量化格點上，這個運算是冪等的）;真正會累積失真的，是每次改變品質設定、或在存檔之間裁切縮放而打亂了 8×8 的對齊。11.4.3 節會完整推導，圖 11-14 則用二十個世代的實測驗證這件事。



量化之後，8×8 區塊的右下角幾乎全是零。問題是怎麼把二維的零「攤成一維」才最省。Zigzag 掃描從左上角（低頻）走到右下角（高頻），恰好把零自然聚在尾端。實測這張影像的全部 1,024 個區塊：零的連續長度平均由逐列掃描的 15.2 提高到 19.6;更關鍵的是最後那一串——zigzag 平均有 51.1 個零可以用一個 EOB 符號直接帶過，逐列掃描只有 43.0 個，相差 8.0 個係數。這一步完全不丟資訊，純粹是把同樣的東西排得更好寫。

圖 11-12　Zigzag 掃描：把二維的零聚成一維的連續零

程式 11-3 DCT 量化與 JPEG 品質實驗

```
import cv2, numpy as np

gray = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE).astype(np.float32)

# --- 手動示範 JPEG 的核心:8x8 區塊 DCT 與量化 ---
# JPEG 標準亮度量化表(quality≈50)
Q = np.array([
    [16,11,10,16,24,40,51,61], [12,12,14,19,26,58,60,55],
    [14,13,16,24,40,57,69,56], [14,17,22,29,51,87,80,62],
    [18,22,37,56,68,109,103,77], [24,35,55,64,81,104,113,92],
    [49,64,78,87,103,121,120,101],[72,92,95,98,112,100,103,99]],
    dtype=np.float32)

block = gray[:8, :8] - 128            # 置中
dct_block = cv2.dct(block)            # 8x8 DCT
```

```
quantized = np.round(dct_block / Q)   # 量化(失真在此發生)
print("非零係數數量:", np.count_nonzero(quantized), "/ 64")

# 反量化 + 反 DCT 還原
recovered = cv2.idct(quantized * Q) + 128

# --- 實用:直接用不同品質存 JPEG,觀察方塊效應 ---
img = cv2.imread("photo.jpg")
for q in [90, 50, 10]:
    cv2.imwrite(f"jpeg_q{q}.jpg", img, [cv2.IMWRITE_JPEG_QUALITY, q])
    back = cv2.imread(f"jpeg_q{q}.jpg")
    print(f"品質 {q}: PSNR = {cv2.PSNR(img, back):.2f} dB")
    # 放大 q10 的結果,可清楚看到 8x8 方塊邊界
```

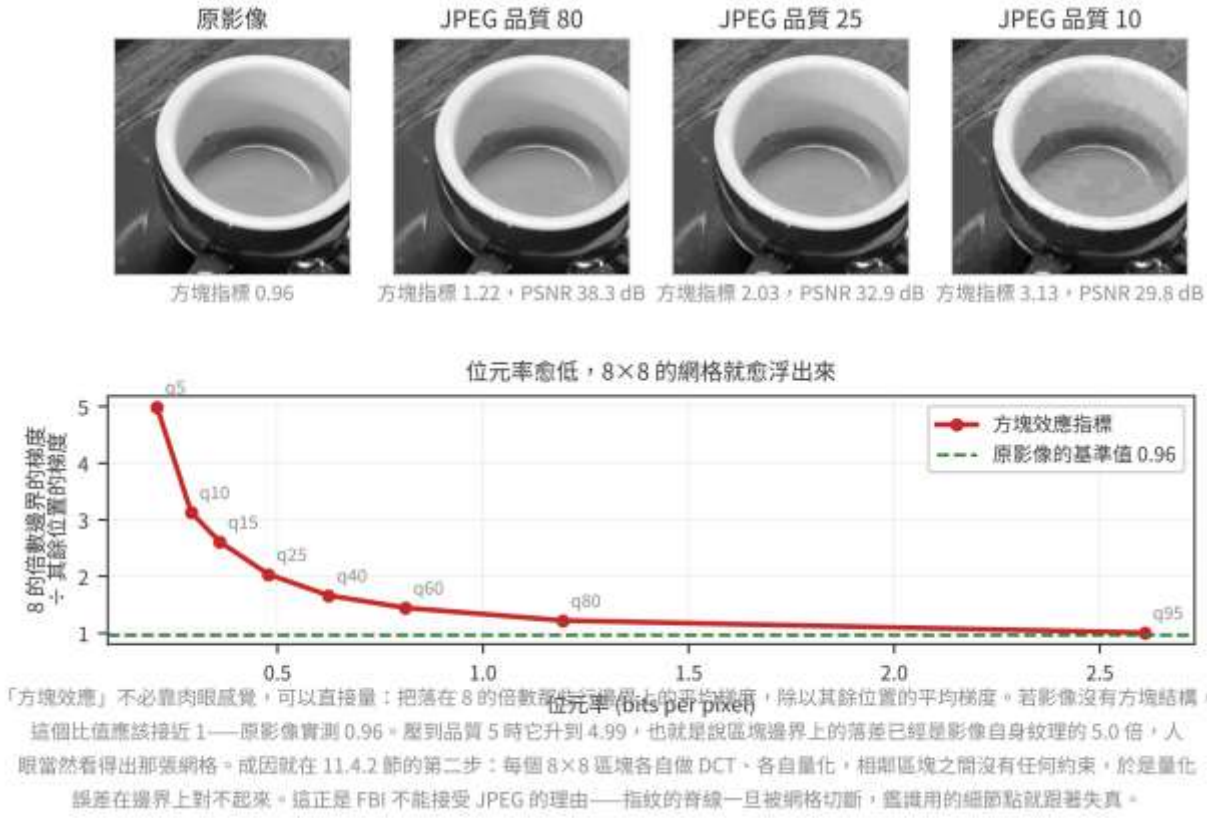


圖 11-13　方塊效應：量化步長變大，格線就浮現

11.4.3 推導：再量化為什麼近乎冪等——「愈存愈糊」的條件

「JPEG 每存一次就糊一次」是流傳極廣的說法，本章 11.4.2 節也提過。但這句話需要補上條件，否則它是錯的。先把量化這個動作寫清楚：編碼時算 $q = \text{round}(F / Q)$，解碼時算 $F' = q \times Q$。合起來看，一次「存檔再打開」做的事是 $F \to \text{round}(F/Q) \times Q$，也就是把係數 F 吸附到最近的 Q 的整數倍上。

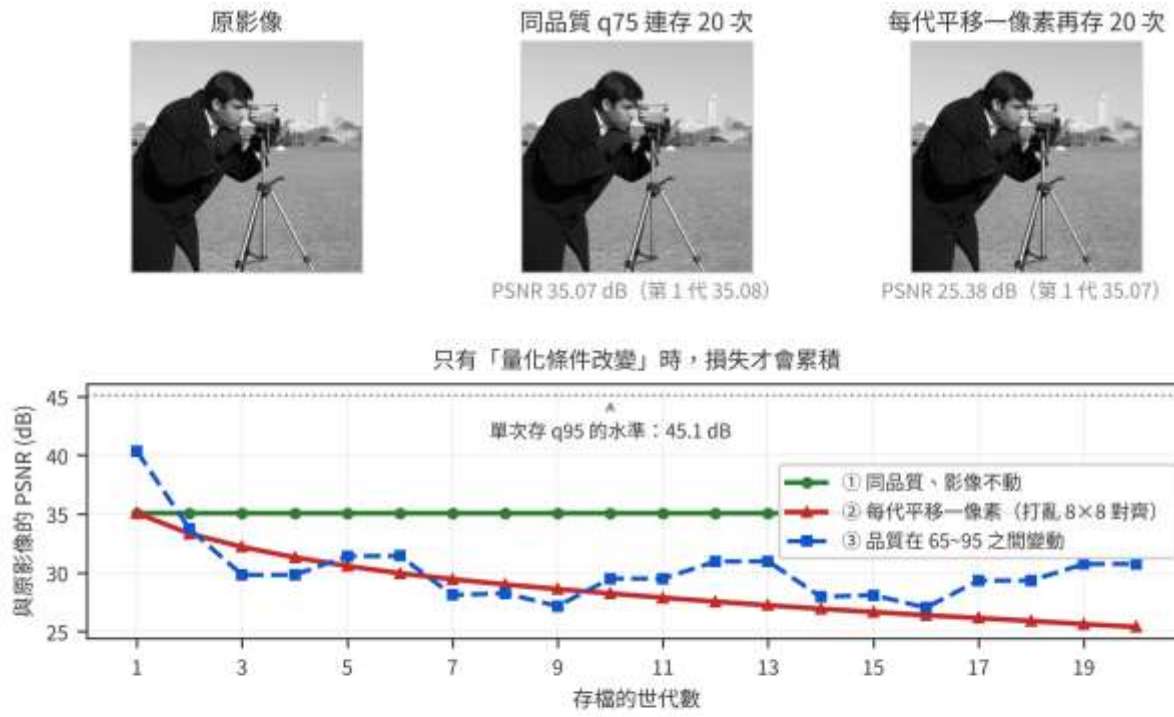
關鍵問題：如果把 F' 再存一次會怎樣?算一下 $\text{round}(F'/Q) \times Q$。因為 F' 已經是 Q

的整數倍，F'/Q 本來就是整數，round 不會改變它，再乘回 Q 自然還是 F'。用數學的話說，這個運算滿足 T(T(F)) = T(F)，它是冪等的（idempotent）——與第 10 章開運算、閉運算的冪等性是同一個概念。所以在 DCT 係數這一層，第二次以後的存檔完全不再損失任何資訊。

這個推論可以用實測檢驗。以同一個品質 q75 連續存 20 次，PSNR 從第 1 代的 35.08 dB 到第 20 代的 35.07 dB，幾乎原地不動。之所以不是分毫不差，是因為解碼還要做反 DCT、取整數、再截斷到 0~255 這三道手續，這些才是微小擾動的來源——第 2 代與第 1 代之間只有約 0.5% 的像素有差異，之後就收斂了。

那麼什麼時候才真的會「愈存愈糊」?答案是：當量化的條件改變時。第一種情況是每次改變品質設定，等於每次換一張不同的量化表 Q，把係數搬到另一個格點上，冪等性立刻失效。圖 11-14 的實測中，品質在 65~95 之間輪流變動存 20 代後只剩 30.8 dB，而且那一代其實是用最高的 q95 存的——單次用 q95 可以達到 45.1 dB。這說明已經丟掉的資訊，事後調高品質救不回來。

第二種情況更常見也更隱蔽：在存檔之間裁切、縮放或旋轉影像。這些操作會讓 8×8 的切割線落在不同的像素位置上，同一塊內容每次被切進不同的區塊、配到不同的鄰居，自然每次都要重新量化一次。圖 11-14 中「每代平移一個像素」的那條線在 20 代之內掉了 9.7 dB，正是這個效應。對實務的建議因此很具體：編修影像時用無失真格式（PNG、TIFF）當工作檔，只在最後輸出時存一次 JPEG;若必須反覆存 JPEG，至少要固定品質設定，而且不要在中途裁切或縮放。



「JPEG 反覆存檔會愈存愈糊」這句話需要補上條件。綠線是情境①：同一個品質連存 20 次，PSNR 從 35.08 dB 到 35.07 dB，幾乎原地不動。原因很乾淨：第一次量化之後，係數已經落在量化格點上，再除以同一張量化表、再取整數就是它自己——近乎冪等（之所以不是完全相等，是因為反轉換後還要取整數並截斷到 0–255，第 2 代只有 1,237 個像素、也就是 0.5% 與第 1 代不同，之後就收斂了）。真正會累積損失的是另外兩條線：②每代平移一像素，8×8 的切割線每次落在不同位置，20 代掉了 9.7 dB；③每代換一個品質，係數每次被搬到不同的格點上。③ 的第 20 代其實是用 q95 存的，單次用 q95 可以達到 45.1 dB，它卻只有 30.8 dB——這說明已經丟掉的資訊，事後把品質調高是救不回來的。結論：反覆存檔本身不可怕，可怕的是中間裁切、縮放或改變品質設定。

圖 11-14　反覆存檔真的會愈存愈糊嗎：三種情境的實測

11.5 小波壓縮與 JPEG 2000

理解了 JPEG 的方塊之痛，小波壓縮的優勢就不言自明。JPEG 2000 是小波壓縮的標準化成果，它用二維離散小波轉換取代了 JPEG 的分塊 DCT。關鍵差異在於：小波轉換作用於整張影像(或大的圖塊)，不把影像硬切成獨立的 8×8 方塊，因此從根本上不存在方塊邊界——高壓縮率下，JPEG 2000 的影像是整體變得柔和模糊，而非碎裂成磁磚，視覺上遠優於同壓縮率的 JPEG。

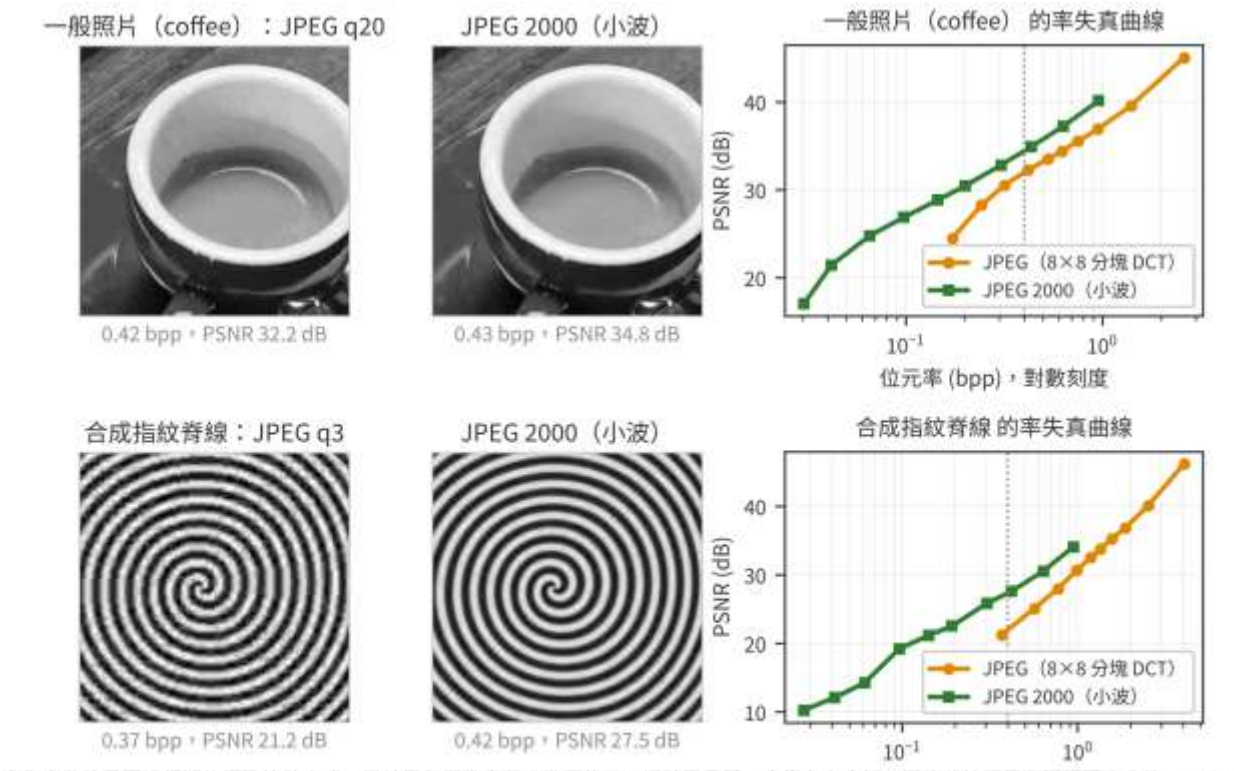
小波壓縮的一般流程與 WSQ、JPEG 2000 一致：對影像做多層小波分解 → 對各子帶的係數做量化(利用能量集中於 LL 子帶、其餘子帶稀疏的特性)→ 熵編碼。除了避免方塊效應，小波壓縮還有一個附加優點：天然支援漸進式傳輸與多解析度存取——先傳最頂層的 LL(縮略圖)就能顯示模糊的全貌，隨著更多子帶傳入逐步變清晰，非常

適合網路傳輸與大圖瀏覽。

比較面向	JPEG(DCT)	JPEG 2000 / WSQ(小波)
轉換單位	8×8 獨立方塊	整張影像多解析度分解
高壓縮率瑕疵	方塊效應、蚊狀雜訊	整體柔化模糊，無方塊
漸進式傳輸	有限(progressive 模式)	天然支援，多解析度
運算複雜度	較低，普及極廣	較高
典型應用	一般照片、網頁、相機	指紋(WSQ)、醫療、電影數位母帶

表 11-1 JPEG 與小波壓縮的比較

那為什麼今天絕大多數照片仍是 JPEG?因為 JPEG「夠好、夠快、夠普及」——它的運算簡單、硬體支援完善、相容性無敵。JPEG 2000 雖然技術更優，卻因複雜度較高與相容性問題，主要用在對品質要求極高的專業領域(醫療影像、電影數位母帶、指紋鑑識)。這是工程世界的常態：最好的技術不一定勝出，「足夠好加上生態系」往往才是贏家。這個觀察值得每一位工程師記在心裡。



率失真曲線是壓縮領域的標準比較方式：橫軸是每個像素花了幾個位元，縱軸是品質，愈靠左上愈好。兩條曲線都內插到同一個位元率 0.40 bpp 再比：一般照片上 JPEG 2000 比 JPEG 高 +2.4 dB（31.9 → 34.3），指紋脊線上高 +5.4 dB（21.8 → 27.3）。差距在高頻密集的指紋上明顯更大，因為分塊 DCT 的量化誤差會沿著 8×8 的格線堆積（圖 11-13），而脊線正好被那些格線切斷。這就是 WSQ 與 JPEG 2000 選小波的量化依據，也說明了為什麼日常照片用 JPEG 就夠——那個差距在一般照片上小得多，而 JPEG 的編碼速度與相容性遠勝一籌。

圖 11-15　率失真曲線：JPEG 與 JPEG 2000 的實測對決

11.6 小波的其他應用

11.6.1 小波去雜訊

小波提供了一種優雅的去雜訊思路，與第 7 章的方法互補。核心觀察是：自然影像的重要結構(邊緣、輪廓)在小波分解後，會集中成少數幾個「大係數」;而雜訊則均勻散布成大量「小係數」。因此去雜訊的策略非常直觀——小波門檻化(Wavelet Thresholding)：把小波係數中絕對值小於某門檻的通通歸零(視為雜訊)，保留大係數(視為訊號)，再反轉換。

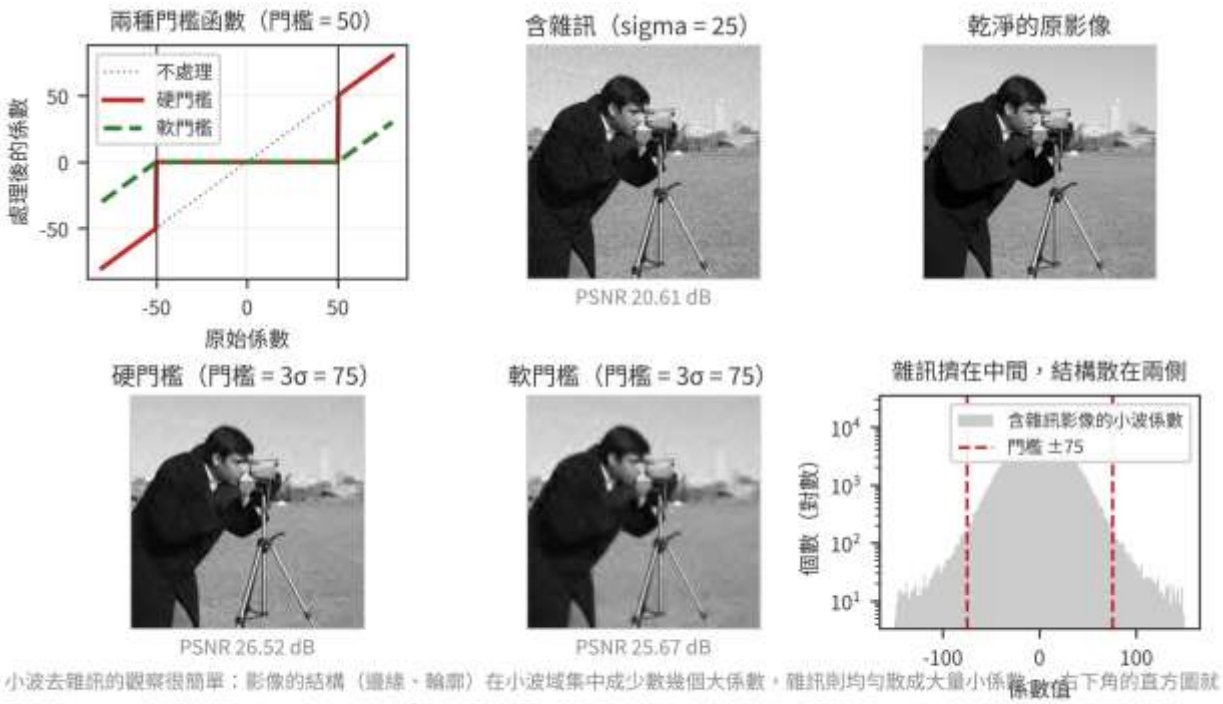
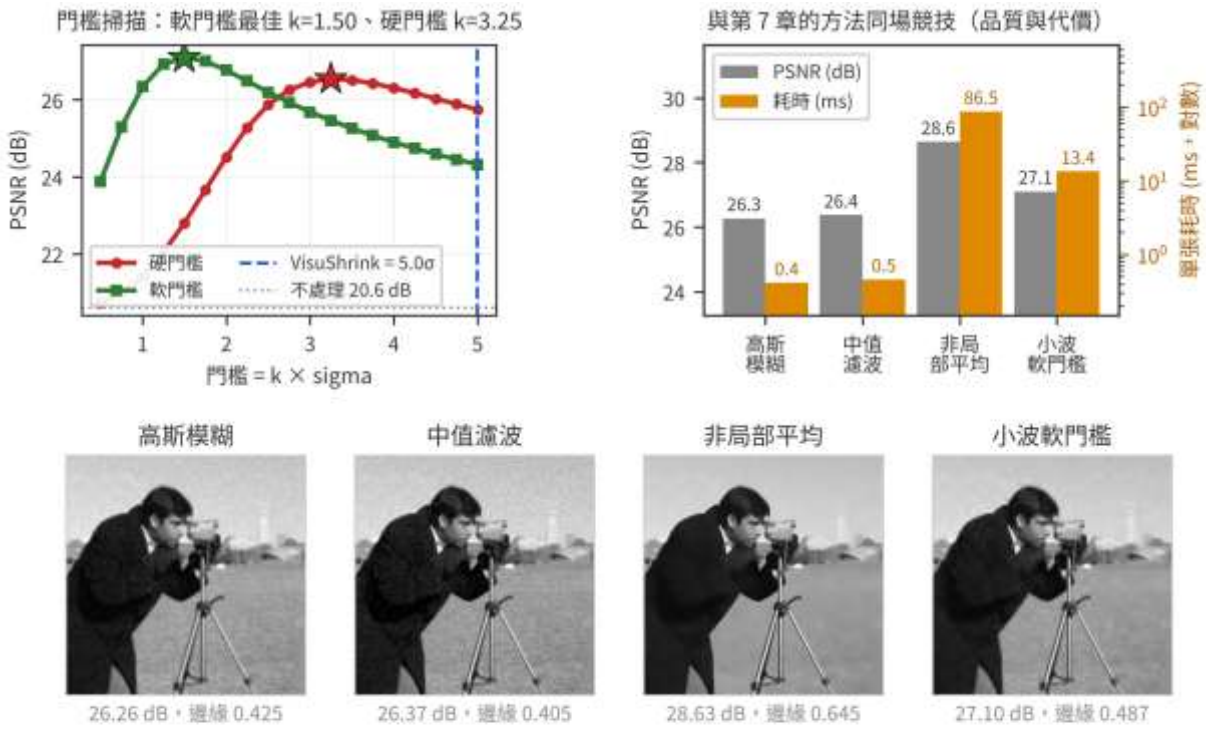


圖 11-16　硬門檻與軟門檻：兩種收縮函數與去雜訊結果

這個方法之所以強大，是因為它在「同時掌握頻率與位置」的小波域中操作：它能區分「邊緣附近的高頻(該保留)」與「平坦區的高頻(是雜訊，該去除)」——這是第 7 章純空間域或第 6 章純頻率域方法難以做到的。它與第 7 章的非局部平均、維納濾波各擅勝場，共同構成去雜訊的工具箱。

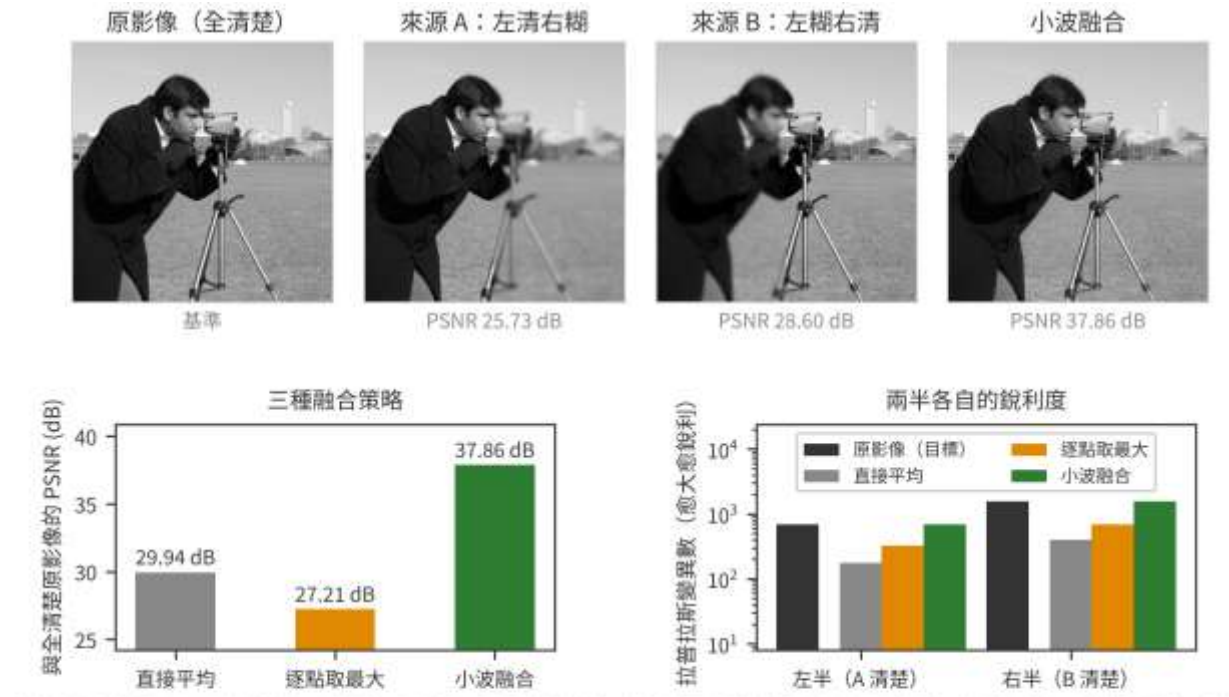


門檻值不能隨便挑。左上掃描 k（門檻 = k×sigma）：軟門檻在 k=1.50 達到最佳 27.10 dB，硬門檻在 k=3.25 達到 26.54 dB——各自調到最好之後，軟門檻反而勝出，這修正了上一張圖的印象。教科書常引用的 VisuShrink 門檻在這裡相當於 5.0σ，只有 24.32 dB，比最佳值低 2.78 dB——它的設計目標是「幾乎不留任何雜訊」，代價是也抹掉細節。右上與下排是誠實的對照：PSNR 最高的不是小波，是非局部平均的 28.63 dB，邊緣保留 0.645 也是它最高。但它要 86.5 ms，是小波的 6.5 倍，而且只會做去雜訊這一件事。小波的價值不在單項奪冠，而在同一套分解可以接著做壓縮、融合與浮水印（接下來三張圖）——這是「換一個基底」帶來的通用性。

圖 11-17　門檻值怎麼挑，以及與第 7 章方法的同場競技

11.6.2 影像融合

影像融合(Image Fusion)把多張影像的資訊合併成一張。典型應用：把一張對焦近物、一張對焦遠物的照片融合成「全景深」清晰影像;把紅外線影像與可見光影像融合以兼得夜視與細節;醫學上把 CT(骨骼)與 MRI(軟組織)融合。小波是影像融合的常用工具：先把各影像做小波分解，再依規則合併各子帶的係數(例如細節子帶取絕對值較大者，代表該處較清晰)，最後反轉換得到融合影像。



多焦點融合的任務：兩張照片各有一半清楚，要合成一張全部清楚的。本圖的兩張來源由同一張清晰影像人工製造（左右各模糊一半），所以有「正確答案」可以比對。直接平均只有 29.94 dB——兩邊都半糊，這是最糟的做法，卻也最常被想到。逐點取最大 27.21 dB，更差，因為它會把亮點誤判成細節。小波融合 37.86 dB：它在每個子帶的每個位置，比較兩張影像誰的細節係數較強就選誰，等於在「頻率 × 位置」的網格上逐格做選擇——這正是小波同時握有兩種資訊才做得到的事。右下角是驗證：融合後左半的銳利度 691（原影像 691）、右半 1579（原影像 1576），兩半都回到了原影像的水準。

圖 11-18　多焦點影像融合：在頻率與位置的網格上逐格選擇

11.6.3 數位浮水印

數位浮水印(Watermarking)把識別資訊(版權標記、序號)隱藏進影像。把浮水印嵌入小波域的中頻係數，是常見的強健做法：它既不像嵌入低頻那樣容易被肉眼察覺，也不像嵌入高頻那樣容易被壓縮或濾波破壞，能在影像被 JPEG 壓縮、縮放後依然存活。這在版權保護與影像鑑識中很重要。不過這個「中頻最好」的通則值得用實驗檢驗一次：若把三個位置的嵌入強度調到隱蔽性完全相同再比較，低頻其實在各種攻擊下全面勝出，因為壓縮、模糊、縮放本質上都是低通運算。低頻真正的限制是容量——它的係數個數比最細的子帶少了十幾倍。換句話說，真正的取捨是「強健度對容量」，圖 11-19 有完整的對照。

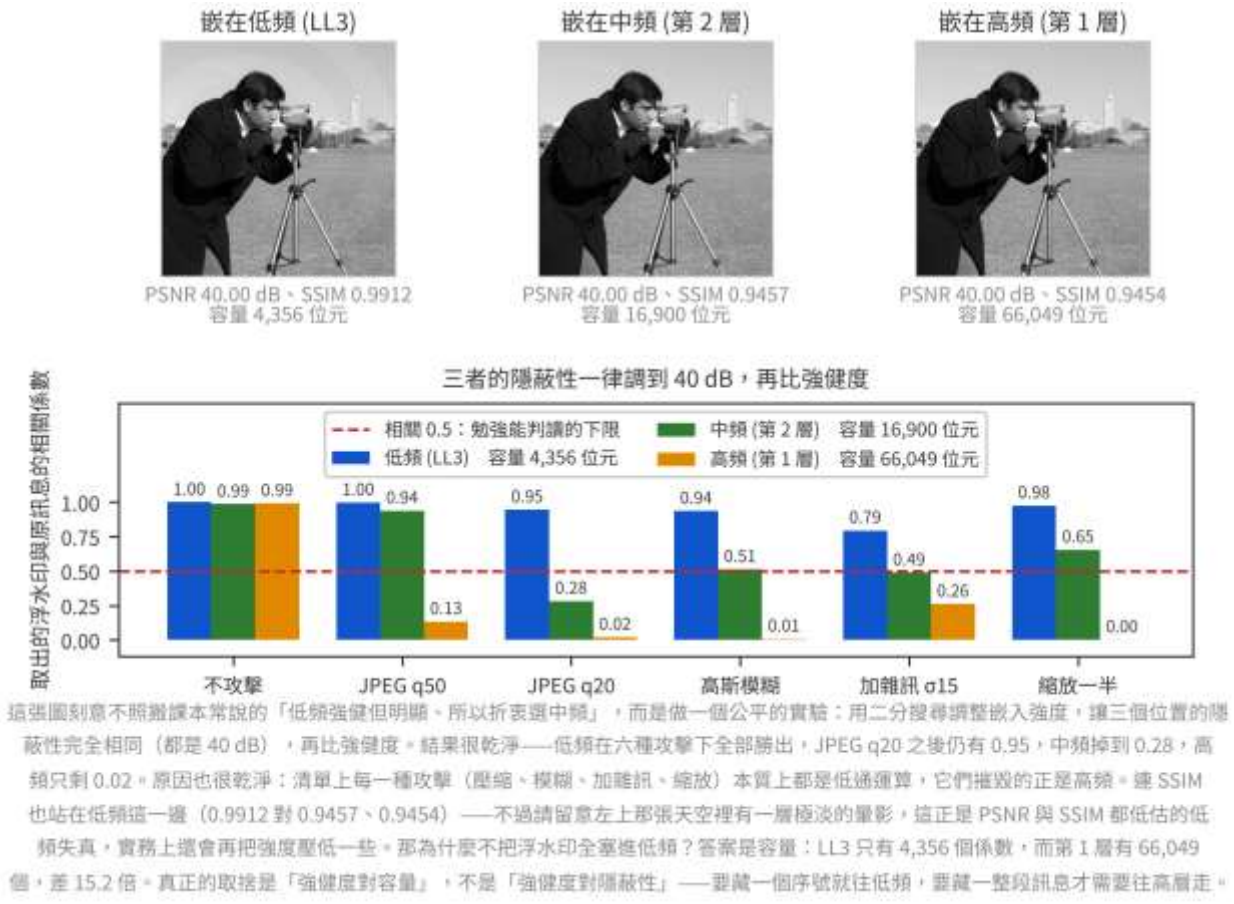


圖 11-19　浮水印嵌在哪一層：等隱蔽性下的強健度與容量

11.6.4 推導：軟門檻為什麼要再收縮一個門檻值

11.6.1 節說小波去雜訊是「把小於門檻的係數歸零」，這是硬門檻。軟門檻多做一件看起來很奇怪的事：留下來的大係數也要再往零減去一個門檻值 T，寫成公式是 soft(w) = sign(w)·max(|w| – T, 0)。明明判定它是訊號了，為什麼還要削它一刀?

理由要從「被觀測到的係數是什麼」說起。影像加了標準差 σ 的高斯雜訊之後，因為小波轉換是正交的（這正是 11.3.3 節推導的 1/√2 在保證的事），雜訊在小波域仍然是標準差 σ 的高斯雜訊，而且均勻散布在每一個係數上。所以觀測到的係數是 w = s + n，其中 s 是真正的訊號、n 是雜訊。我們想要的是 s，手上只有 w。

現在看硬門檻做了什麼。當 |w| > T 時它原封不動地保留 w，也就是把 s + n 當成 s 用——雜訊 n 整個被留了下來。門檻只擋掉了「純雜訊造成的假訊號」，卻沒有處理「真訊號上附帶的雜訊」。軟門檻的 –T 正是針對這一項：既然每個留下的係數都

夾帶著雜訊，就統一把它往零拉回一段，平均而言可以抵消掉一部分雜訊的貢獻。

代價是引入偏差（bias）。對一個真實強度很大的係數——比如一條強邊緣，w = 300、T = 75——軟門檻會把它變成 225，硬是削掉了四分之一，邊緣因此變淡。所以軟門檻是拿「偏差」去換「變異數」：係數整體變得平滑穩定，但系統性地偏小。硬門檻反過來，不帶偏差卻保留了全部雜訊，而且在門檻附近有跳躍，反轉換後會在影像上留下細小的偽影。

既然兩者的取捨不同，它們的最佳門檻自然也不同，比較時必須各自調到最好才公平。圖 11-16 用同樣的 3σ 門檻比較，硬門檻（26.52 dB）看似勝過軟門檻（25.67 dB）;但圖 11-17 把門檻掃過一遍之後，軟門檻在 1.50σ 達到 27.10 dB，反而勝過硬門檻在 3.25σ 的 26.54 dB。軟門檻的最佳門檻明顯較低，原因就在上一段：它本來就會額外削掉一截，門檻再設高就削過頭了。這也提醒一件更一般的事——比較兩個方法時，若只在同一組參數下較量，得到的往往是參數的高下，不是方法的高下。

11.7 小波在深度學習時代的定位

進入深度學習時代，壓縮與特徵擷取的許多工作逐漸被神經網路接手，小波是否過時了?答案與第 9、10 章一致：它轉變了角色，但並未退場。

三個觀察值得學生留意。其一，小波仍是專業壓縮的中流砥柱：指紋(WSQ)、醫療影像、電影數位母帶仍大量使用小波，因為這些領域對「無方塊、可鑑識」的要求無可妥協。其二，小波與深度學習正在融合：近年出現不少「小波卷積網路」的研究，把小波的多解析度分解作為神經網路的一層，結合兩者的長處——用小波提供的多尺度結構，幫助網路更有效率地學習。其三，多解析度的思想無所不在：第 13 章 SIFT 的尺度空間、卷積網路中的多層降取樣(每一層看到的都是更粗尺度的特徵)，骨子裡都與小波「由粗到細、多尺度分析」的精神相通。

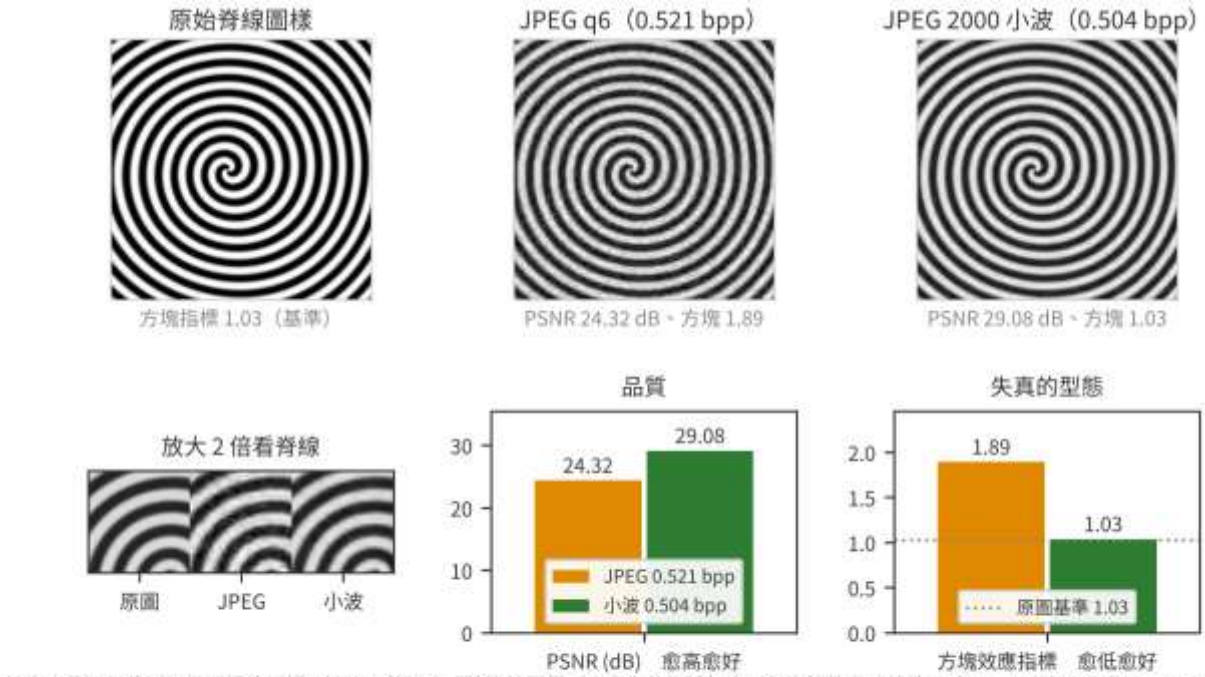
換言之，小波作為一個「具體演算法」或許在日常應用中不如深度學習搶眼，但

它所承載的「多解析度分析」思想，已經深深融入現代電腦視覺的血液。理解小波，就是理解這個貫穿全書(第 6 章頻率、第 9 章尺度、第 13 章 SIFT、第 15 章 CNN)的多尺度思維。

11.8 應用案例

11.8.1 案例一：指紋壓縮與 WSQ

呼應叫獸開講，這是小波壓縮的旗艦案例。情境：執法機構需要儲存與傳輸海量的 500 ppi 指紋影像，且不能損失用於鑑識的脊線細節。WSQ 的流程：對整張指紋影像做多層離散小波分解(精選適合指紋脊線頻率特性的小波)→ 對各子帶做純量量化(依子帶的重要性分配不同的量化精度，脊線所在的頻帶量化得細)→ Huffman 熵編碼。在 20:1 壓縮率下，人眼幾乎無法察覺品質損失，關鍵的特徵點得以保留。這個案例完美展示了小波「無方塊、保細節」相對 JPEG 的決定性優勢——在鑑識這種細節攸關人命的場合，方塊效應是不可接受的。



把本章的工具放回叫獸開講的現場。這裡用的是合成的脊線圖樣（真實指紋屬於個資，但脊線的空間特性一致）。兩者都壓到約 0.5 bpp、也就是約 16 倍：JPEG 用了 0.521 bpp 換到 24.32 dB，小波用更少的 0.504 bpp 換到 29.08 dB——位元數較少卻贏了 4.76 dB。更要命的是失真的「型態」：JPEG 的方塊指標 1.89 遠高於原圖基準 1.03，而小波的 1.03 幾乎沒變，左下角放大圖也看得見 JPEG 把脊線切成一段一段。對鑑識而言，脊線的分岔點與端點就是比對依據，一旦被 8×8 網格切斷就可能生出假的細節點——這是精度問題，更是證據可信度的問題。FBI 選小波（WSQ）不是因為它時髦，而是因為 JPEG 的失真型態正好落在最不能失真的地方。

圖 11-20　WSQ 的理由：同一位元率下，失真的型態才是關鍵

11.8.2 案例二：多焦點影像融合

情境：用顯微鏡或微距鏡頭拍攝時，景深很淺，一張照片只能拍清楚某個深度的物體。要得到「全部清晰」的影像，可拍攝一系列對焦在不同深度的照片，再融合。小波做法：對每張照片做小波分解 → 在每個位置，細節子帶取「絕對值最大」的係數(絕對值大代表該處有清晰的邊緣細節，即該張在此處對焦準確)→ 反轉換得到處處清晰的融合影像。這在顯微攝影、微距攝影、工業檢測(檢查有高低起伏的工件)中極為實用。

11.8.3 案例三：小波去雜訊對照第 7 章

情境：同一張含高斯雜訊的影像，分別用第 7 章的方法(高斯、中值、非局部平均、維納)與本章的小波門檻化去雜訊，比較成效。小波法的特色在於它對「邊緣的保留」通常較佳——因為它能在小波域中區分「邊緣的高頻(大係數，保留)」與「雜訊的高頻(小係數，歸零)」，而空間域的高斯模糊會無差別地抹平兩者。以 PSNR 與 SSIM 量化，並附上邊緣區域的局部放大，學生能清楚看到不同方法的取捨。這個案例把第 7 章與第 11 章的去雜訊工具箱完整地擺在一起比較。

11.8.4 案例四：壓縮率與品質的權衡實驗

情境：對同一張影像，分別以不同壓縮率存成 JPEG 與 JPEG 2000(或用 PyWavelets 模擬小波壓縮)，繪製「壓縮率對 PSNR/SSIM」的率失真曲線(rate-distortion curve)，並在高壓縮率處並排比較兩者的視覺瑕疵。學生會看到：低壓縮率時兩者差異不大，但當壓縮率推到極高(如 50:1),JPEG 碎裂成明顯的方塊，而小波僅是整體柔化——這正是叫獸開講中 FBI 捨 JPEG 就小波的量化證據。這個案例也是理解「率失真」這個壓縮領域核心概念的最佳實作。

11.9 本章與全書的觀念連結總表

相關章節	共用的觀念	本章的延伸或差異
第 3 章 數位影像基礎	位元深度、PSNR/SSIM、檔案格式	壓縮的品質以 PSNR/SSIM 量化;RAW 對照壓縮格式
第 5 章 空間域強化	平滑去雜訊	小波門檻化是另一種去雜訊途徑
第 6 章 頻率域	換基底、頻率、測不準	小波補上傅立葉缺的位置資訊(多解析度)
第 7 章 影像還原	去雜訊工具箱	小波門檻化與維納、NLM 互補，較能保邊緣
第 8 章 色彩處理	YCbCr、色度降取樣	JPEG 第一步即 YCbCr 轉換與 4:2:0 降取樣
第 9 章 影像分割	尺度空間	多解析度分析與 LoG/DoG 的尺度思想相通
第 13 章 特徵擷取	SIFT 尺度空間	SIFT 的多尺度金字塔與小波多解析度同源
第 15 章 深度學習	多層降取樣、多尺度特徵	CNN 的層級降取樣與小波多解析度精神相通

表 11-2 第 11 章與全書其他章節的觀念連結

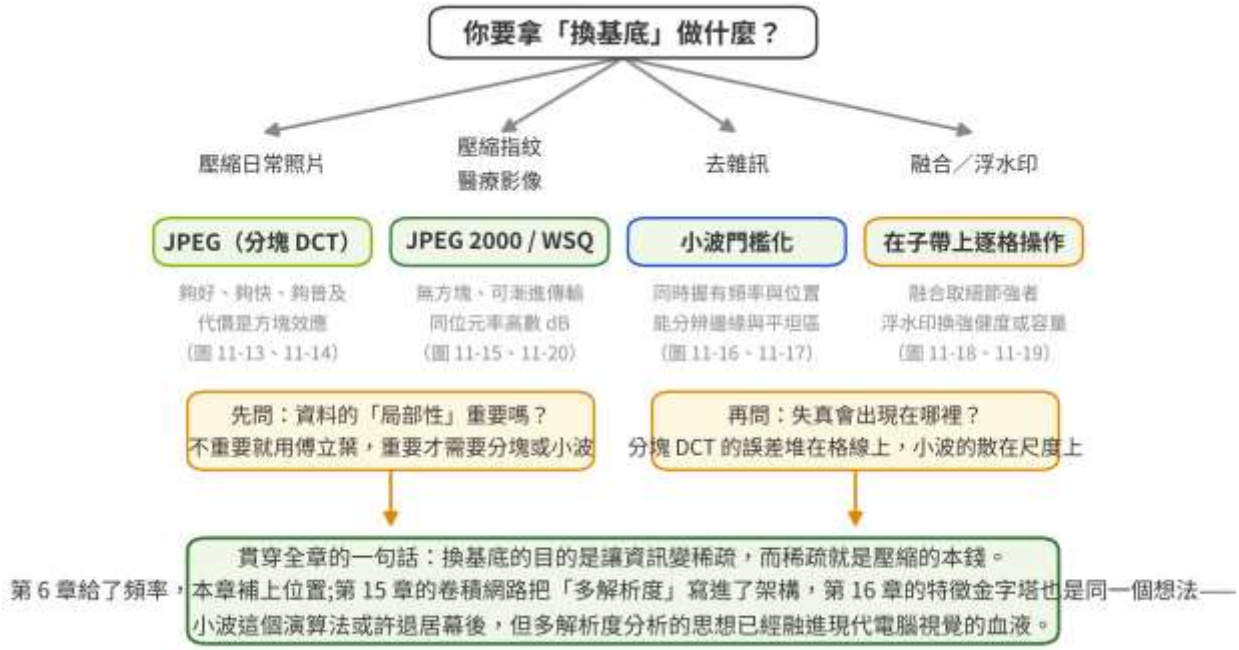


圖 11-21　第 11 章的決策地圖

程式實作 Lab 11

Lab 11-1 手算 Haar 小波

目標：用最簡單的方式理解小波。(a)不用任何函式庫，對一維數列 [8, 4, 6, 10, 12, 8, 4, 2] 手算(或用 Python 純運算)一層 Haar 分解，得到 4 個概略值與 4 個細節值，並驗證能無失真還原;(b)繼續對概略序列做第二、三層分解，觀察多解析度結構;(c)把最小的細節係數歸零後還原，計算與原數列的誤差——體會「丟棄小係數」的壓縮思想;(d)用 PyWavelets 的 pywt.wavedec 驗證你的手算結果。

Lab 11-2 影像的小波金字塔

目標：視覺化二維小波分解。(a)以程式 11-2 對一張影像(建議用指紋圖或紋理豐富的圖)做單層分解，把 LL、LH、HL、HH 四個子帶拼成一張圖顯示，說明每個子帶凸顯了什麼方向的結構;(b)做三層分解，顯示金字塔結構，觀察能量如何集中在頂層 LL;(c)統計各子帶中「絕對值小於某門檻」的係數比例，驗證細節子帶的稀疏性;(d)只保留最大的 5%、10%、25% 係數，其餘歸零後重建，計算各自的 PSNR——這就是

小波壓縮的核心實驗。

Lab 11-3 JPEG 方塊效應解剖

目標：親眼見證叫獸開講的方塊效應。(a)以程式 11-3 對一張影像用品質 90、50、30、10 分別存成 JPEG，計算各自的 PSNR 與檔案大小，繪製率失真曲線;(b)把品質 10 的結果放大到像素級，清楚標出 8×8 方塊邊界與蚊狀雜訊;(c)手動實作單一 8×8 區塊的 DCT 與量化，統計量化後非零係數的數量，理解壓縮如何發生;(d)找一張含銳利邊緣(如文字)的影像，觀察高壓縮率下邊緣附近的蚊狀雜訊——說明它與第 6 章振鈴效應的關聯。

Lab 11-4 小波去雜訊擂台

目標：把第 7 章與第 11 章的去雜訊方法同場競技。(a)對一張乾淨影像加入已知強度的高斯雜訊;(b)分別用高斯模糊、中值、非局部平均(第 7 章)與小波門檻化(本章)去雜訊;(c)以 PSNR 與 SSIM 製作比較表，並附上「邊緣區域局部放大」的對照圖;(d)特別觀察：小波法在「邊緣銳利度」與「平坦區乾淨度」的平衡上表現如何?為什麼它能同時做到(提示：在小波域區分邊緣高頻與雜訊高頻)?(e)討論：什麼情況下你會選小波去雜訊，什麼情況下選非局部平均?

Lab 11-5 反覆存檔的真相

目標：親手推翻（或補上條件給）一個流傳很廣的說法，並練習設計對照實驗。(a)取一張灰階影像，用同一個品質 q75 連續「存檔再讀回」20 次，每一代都與最初的原影像計算 PSNR，畫出世代—PSNR 曲線;(b)統計第 1 代與第 2 代之間有多少像素不同，佔全部像素的百分比是多少?試從 round(F/Q)×Q 的冪等性解釋這個數字為什麼這麼小，以及它為什麼不是零（提示：反 DCT 之後還要取整數與截斷到 0~255）;(c)改為每一代存檔前先把影像平移一個像素（且不移回），與同樣平移量的原影像比對 PSNR，重畫曲線;(d)再改為每一代輪流使用 90、80、70、85、75、95、65 的品質，重畫曲線。注意第 20 代其實是用 q95 存的，請與「對原影像單次存 q95」

的 PSNR 比較，並解釋這個落差說明了什麼;(e)把三條曲線畫在同一張圖上，寫出你會給攝影同好的一句具體建議。

Lab 11-6 浮水印的公平擂台

目標：學會設計「控制變因」的比較實驗——這是本章最重要的方法論訓練。(a)以 db2 做 3 層分解，分別把同一個 ±1 的二值浮水印嵌入 LL3（低頻）、第 2 層與第 1 層的細節子帶（中頻、高頻），嵌入方式為「係數 += α × 浮水印 × 該子帶係數絕對值的平均」;(b)先用固定的 α 比較三者，記錄各自與原影像的 PSNR——你應該會發現三者差距極大，這樣比較強健度是不公平的;(c)改用二分搜尋調整每個位置的 α，讓三者的 PSNR 都等於 40 dB;(d)對三張浮水印影像施加六種攻擊：不攻擊、JPEG q50、JPEG q20、高斯模糊 σ=1.2、加高斯雜訊 σ=15、縮放到一半再放大回來。每次都取出浮水印並與原訊息計算相關係數，畫成分組長條圖;(e)你的結果支持還是推翻「浮水印應該嵌在中頻」這個常見建議?請同時報告三個子帶的係數個數（容量），並用「強健度對容量」重新表述這個取捨;(f)觀察低頻那張影像的平坦區（例如天空），你看得到 PSNR 與 SSIM 都沒反映出來的淡暈影嗎?這說明了什麼?

本章重點整理

- 正交轉換的共同思想是「換一組基底表示影像」，使能量集中於少數係數(稀疏)，而稀疏是壓縮的黃金鑰匙;正交性帶來可逆、能量守恆、係數互不干擾三大好處。
- 傅立葉的根本弱點是「有頻率、無位置」(全域基底);短時傅立葉以固定窗口取得局部性但受測不準限制;小波以縮放與平移實現「低頻用長窗、高頻用短窗」的自適應多解析度分析，同時掌握頻率與位置。
- Haar 小波以「相鄰值的平均與差分」做分解，可手算;二維小波一層產生 LL(縮略圖)、LH、HL、HH 四個子帶，多層時只續分 LL，形成能量集中於頂層的金字塔。

- DCT 只用餘弦基底、係數為實數、能量集中;JPEG 五步驟為 YCbCr 降取樣、8×8 分塊 DCT、量化(唯一失真來源)、zigzag 與遊程編碼、熵編碼;反覆存檔只有在「改變品質設定」或「中途裁切縮放」時才會累積失真，同品質且不動影像的再存檔近乎冪等（11.4.3 節）。
- 小波壓縮(JPEG 2000、WSQ)作用於整張影像不切塊，從根本避免方塊效應，並天然支援漸進式與多解析度存取;但 JPEG 因夠好、夠快、夠普及而仍是日常主流。
- 小波應用：門檻化去雜訊(能區分邊緣高頻與雜訊高頻，較保邊緣)、多焦點/多模態影像融合、中頻域數位浮水印;其「多解析度」思想延續到 SIFT 尺度空間與 CNN 的層級降取樣。

習題

1. 正交轉換的共同思想是什麼?為什麼「換基底使能量稀疏」有利於壓縮?自然影像在空間域是否稀疏?

   > *提示與參考解答：共同思想是把影像從一組基底轉換到另一組基底表示。換到好的基底(頻率域、小波域)後能量集中於少數係數(稀疏)，只需記住少數重要係數、丟棄大量近零係數即可重建，故利於壓縮。自然影像在空間域不稀疏(每個像素都攜帶資訊)，但轉換後變稀疏——這正是轉換的價值。*

2. 「正交」是什麼意思?它為轉換帶來哪三個好處?

   > *提示與參考解答：正交指基底函數彼此內積為零、互相獨立不重疊。三個好處：轉換可逆(能無失真變回原影像)、能量守恆(Parseval 定理，轉換前後總能量不變，可判斷丟棄係數的能量損失)、係數互不干擾(改一個不影響其他)。*

3. 傅立葉轉換的根本弱點是什麼?用一個生活化的比喻說明。短時傅立葉如何補救，又受什麼限制?

   > *提示與參考解答：弱點是「有頻率、無位置」：基底是無限延伸的弦波，只知道有哪些頻率、不知道頻率來自哪個位置。比喻：分析樂曲只能列出用到哪些音符，卻不知先後順序。短時傅立葉把影像切塊各自轉換以取得位置，但受測不準限制：窗*

> *口大則頻率準位置模糊，窗口小則位置準頻率模糊，無法兼得。*

4. 小波如何同時掌握頻率與位置?請說明「低頻用長窗、高頻用短窗」背後的觀察。

   > *提示與參考解答：小波用有限長度的小波形為基底，透過縮放(對應頻率)與平移(對應位置)分析。背後觀察：低頻成分變化緩慢，需長窗才看得清(換取頻率精度);高頻成分變化劇烈短暫，需短窗才能定位(換取位置精度)。小波自動實現這種自適應，即多解析度分析。*

5. 對數對 [8, 4] 做一層 Haar 小波分解，寫出概略值與細節值，並驗證還原。

   > *提示與參考解答：概略值(平均)= (8+4)/2 = 6;細節值(差的一半)= (8-4)/2 = 2。表示為 [6, 2]。還原：8 = 6+2、4 = 6-2，無失真。*

6. 二維小波一層分解產生哪四個子帶?各凸顯什麼?多層分解時只對哪個子帶繼續分解，為什麼?

   > *提示與參考解答：LL(縮略圖，集中能量)、LH(水平邊緣)、HL(垂直邊緣)、HH(對角細節與雜訊)。多層時只續分 LL，因為能量集中在 LL，其餘子帶已稀疏;續分 LL 可得到由粗到細的金字塔，大部分能量集中在最頂層小 LL，其餘近零——這就是壓縮的機會。*

7. 請完整列出 JPEG 壓縮的五個步驟，並指出哪一步是唯一的失真來源。另外：「反覆存檔 JPEG 會愈存愈糊」在什麼條件下成立、什麼條件下不成立?

   > *提示與參考解答：一、YCbCr 轉換與色度 4:2:0 降取樣;二、8×8 分塊做 DCT;三、量化(唯一失真來源);四、zigzag 掃描與遊程編碼;五、Huffman 熵編碼。量化把係數除以量化步長取整，不可逆。但再量化是冪等的：若每次都用同一張量化表且影像未被移動，係數已在格點上，round(F/Q)×Q 不會再改變它，實測連存 20 次幾乎無損。只有改變品質（換了量化表）或在存檔之間裁切縮放（打亂 8×8 對齊）才會累積失真。詳見 11.4.3 節與圖 11-14。*

8. JPEG 的方塊效應從何而來?小波壓縮(JPEG 2000/WSQ)為什麼能避免它?

   > *提示與參考解答：方塊效應源於 JPEG 把影像切成獨立的 8×8 方塊各自 DCT 與量化，高壓縮率下相鄰方塊的量化差異在邊界顯現。小波轉換作用於整張影像(不切塊)，不存在方塊邊界，高壓縮率下只是整體柔化模糊而非碎裂，故避免方塊效應。*

9. 承叫獸開講：FBI 為什麼不用 JPEG 而選擇小波(WSQ)來壓縮指紋?這對「技術選型」有何啟示?

   提示與參考解答：指紋辨識靠脊線的細微特徵點(分岔、端點),JPEG 的方塊效應與高頻流失會破壞或偽造這些鑑識關鍵特徵，法律上不可接受。小波無方塊、能保脊線細節，在 20:1 壓縮下無感知損失。啟示：技術選型須看應用的核心需求——一般照片能容忍方塊，鑑識則不能;沒有最好的壓縮，只有最適合任務的壓縮。

10. 小波門檻化如何去雜訊?為什麼它在保留邊緣方面常優於空間域的高斯模糊?

    提示與參考解答：自然影像的重要結構在小波域集中成少數大係數，雜訊散布成大量小係數;把小於門檻的小係數歸零(視為雜訊)、保留大係數(視為訊號)，再反轉換。它在「同時掌握頻率與位置」的小波域操作，能區分「邊緣附近的高頻(保留)」與「平坦區的高頻(雜訊，去除)」;高斯模糊則無差別抹平所有高頻，故傷邊緣。

11. 為什麼技術更優的 JPEG 2000 至今仍未取代 JPEG 成為日常主流?這反映了什麼工程通則?

    提示與參考解答：JPEG 運算簡單、硬體支援完善、相容性無敵、「夠好夠快夠普及」;JPEG 2000 複雜度較高、相容性較差，主要用於醫療、電影母帶、指紋等高要求專業領域。通則：最好的技術不一定勝出，「足夠好加上成熟生態系」往往才是贏家(與第 9、10 章「適合勝於最優」呼應)。

12. (綜合題)小波的「多解析度分析」思想，如何呼應第 9 章的尺度空間、第 13 章的 SIFT 與第 15 章的 CNN?

    提示與參考解答：多解析度=由粗到細、多尺度分析。第 9 章 LoG/DoG 在不同尺度找邊緣;第 13 章 SIFT 建高斯尺度金字塔在多尺度找特徵點;第 15 章 CNN 透過層級降取樣，每層看到更粗尺度的特徵。它們與小波「不同尺度各司其職」的精神同源——多尺度思維貫穿全書，是現代電腦視覺的核心觀念之一。

13. 承第 2 題的能量守恆與第 5 題的 [8, 4]：對它做一層 Haar 分解，若用「平均與差的一半」寫成 [6, 2]，轉換前後的能量各是多少?改用 1/√2 的正規化版本呢?由此說明為什麼正交性是 Parseval 定理的前提。

    提示與參考解答：原始能量 $8^2 + 4^2 = 80$。[6, 2] 的能量 = 36 + 4 = 40，少了一半，不滿足 Parseval。1/√2 版本為 [12/√2, 4/√2] ≈ [8.485, 2.828]，能量 = 72 + 8

= 80，與原始相同。把一層 Haar 寫成 H = c·[[1,1],[1,−1]]，HᵀH = 2c²·I，要等於單位矩陣必須 c = 1/√2——1/√2 是正交性唯一允許的值。壓縮之所以敢說「丟掉小係數只損失一點點」，正是因為正交保證了「係數的平方和 = 影像能量」，丟掉大小為 ε 的係數造成的重建誤差能量恰好是 ε²。詳見 11.3.3 節的推導。

14. 測不準原理在短時傅立葉上的具體意義是什麼?若取樣率 fs = 1000 Hz、窗口長度 w = 64 個樣本，時間解析度 Δt 與頻率解析度 Δf 各是多少?兩者的乘積與窗口長度有何關係?

    提示與參考解答：Δt = w / fs = 64/1000 = 0.064 秒;Δf = fs / w = 1000/64 ≈ 15.6 Hz。乘積 Δt·Δf = (w/fs)·(fs/w) = 1，與窗口長度完全無關——這就是測不準的具體形式：窗口拉長換來頻率精度，卻等量地犧牲時間精度，兩者的乘積是一個無法突破的常數。圖 11-4 用三個不同窗長驗證了這一點。小波的回應不是突破這個下限（做不到），而是重新分配：低頻用長窗（頻率準）、高頻用短窗（位置準），讓每個頻段拿到它最需要的那種精度。

15. 二維小波做 N 層分解後，一共有幾個子帶?若原影像為 512×512，做 3 層 db2 分解，係數的總數大約是多少?這對「壓縮」意味著什麼?

    提示與參考解答：N 層分解有 3N + 1 個子帶（每層產生 LH、HL、HH 三個，最後再加一個最頂層的 LL）。3 層就是 10 個子帶。係數總數與原像素數大致相同（邊界延拓會讓實際數字略多一些，例如 512×512 的 db2 三層分解約為 530×530 的量級）。關鍵認識：小波轉換本身「不壓縮任何東西」——它是可逆的換基底，係數不會變少。真正帶來壓縮的是轉換之後的稀疏性：能量集中到最頂層那個小小的 LL，其餘九個子帶的係數大多接近零，量化與熵編碼才有東西可榨。把「轉換」與「壓縮」分開理解，是讀懂 JPEG 與 JPEG 2000 的關鍵。

16. 教科書常說「DCT 比 DFT 更能集中能量」。請說明這個說法成立的真正原因，並指出它在什麼情況下差別很小、什麼情況下差別很大。

    提示與參考解答：原因在邊界，不在基底本身。DFT 隱含假設訊號是週期延拓的，若訊號左右兩端的值差很多，延拓後會在接縫處產生一個人造的跳階，這個不存在的邊緣必須靠大量高頻係數才能描述，能量因此被攤開。DCT 改用「鏡射延拓」，接縫處連續，就沒有這個假邊緣。所以差別的大小取決於訊號兩端的落差：對一條線性斜坡（兩端落差最大），圖 11-10 實測 DFT 需要 25 個係數才能涵蓋 99% 的交

流能量，DCT 只要 2 個;但對自然影像中隨機取出的 32 點列，兩者的中位數都是 22 個，幾乎分不出高下。這也解釋了 JPEG 為什麼選 DCT——它切成 8×8 的小塊，每一塊的邊界到處都是，邊界效應被放大了數千倍。

17. JPEG 的量化表會隨品質因子縮放。已知縮放倍率 s 的定義為：品質 q < 50 時 s = 5000/q，q ≥ 50 時 s = 200 − 2q，量化步長 Q' = floor((Q × s + 50) / 100)（下限為 1）。若標準亮度表的某個位置 Q = 16，請算出 q = 90、q = 50、q = 10 時的實際步長，並說明壓縮率與品質為何不是線性關係。

    提示與參考解答：q = 90 → s = 200 − 180 = 20，Q' = floor((320+50)/100) = 3;q = 50 → s = 100，Q' = floor((1600+50)/100) = 16（即標準表本身）;q = 10 → s = 500，Q' = floor((8000+50)/100) = 80。從 q90 到 q50 步長變 5.3 倍，從 q50 到 q10 再變 5 倍，但 q 的區間一邊是 40、一邊也是 40——可見品質數字與實際量化強度之間是倒數關係，不是線性。這就是為什麼 q95 與 q90 的檔案大小差很多，q20 與 q10 的視覺差異卻遠比數字差距來得劇烈。

18. Zigzag 掃描為什麼能提升壓縮率?若某個 8×8 區塊量化後只有左上角 6 個係數非零，以列優先（row-major）掃描與以 zigzag 掃描，各自能得到多長的尾端連續零?這與 EOB（End of Block）標記有什麼關係?

    提示與參考解答：量化後的非零係數集中在左上角（低頻），zigzag 以之字形由左上往右下掃，使頻率由低到高大致單調遞增，因此所有的零會聚集在序列尾端。列優先掃描則每掃完一列就跳回最左邊（回到低頻），零與非零交錯出現，無法形成長串。有了尾端的長串零，編碼器只要放一個 EOB 標記就能代表「後面全是零」，不必逐一記錄——這是 JPEG 壓縮率的主要來源之一。圖 11-12 用實際區塊比較了兩種掃描的游程長度。

19. 「反覆存檔 JPEG 會愈存愈糊」這句話在什麼條件下成立?什麼條件下不成立?請用量化的數學式說明，並指出實務上該如何避免累積失真。

    提示與參考解答：存檔再打開在係數層面做的是 F → round(F/Q)×Q，也就是把係數吸附到最近的 Q 的整數倍上。若第二次用同一張量化表 Q，因為 F' 已是 Q 的整數倍，F'/Q 是整數、round 不改變它，所以 T(T(F)) = T(F)——這個運算是冪等的（同第 10 章開閉運算的冪等性）。圖 11-14 實測：同品質 q75 連存 20 次，PSNR 僅由 35.08 dB 變成 35.07 dB。不成立的條件有二：一是每次改變品質（換了量化

表），實測在 65~95 間變動存 20 代後只剩 30.8 dB，而該代其實是用 q95 存的（單次 q95 可達 45.1 dB）——已丟的資訊救不回來;二是存檔之間裁切、縮放或旋轉，8×8 的切割線錯位，實測 20 代掉了 9.7 dB。實務建議：工作檔用 PNG/TIFF，最後才輸出一次 JPEG;若必須反覆存，至少固定品質且不在中途裁切縮放。詳見 11.4.3 節。

20. 要比較 JPEG 與 JPEG 2000 的優劣，為什麼不能直接比「同一個品質設定下的 PSNR」?正確的比較方式是什麼?請說明率失真曲線的讀法。

    提示與參考解答：兩種編碼器的「品質」參數定義完全不同（JPEG 是量化表縮放倍率，JPEG 2000 是目標壓縮比），同樣標 50 並不代表花了同樣多的位元，比出來的是參數對照表，不是壓縮效率。正確做法是畫率失真曲線：橫軸是位元率（bpp = 檔案位元數 ÷ 像素數），縱軸是失真指標（PSNR 或 SSIM），讓兩者在「同樣的位元預算」下較量。由於兩條曲線的取樣點不會剛好對齊，報告差距時還要先把兩條曲線內插到同一個 bpp，才能說出「在 0.40 bpp 處相差幾 dB」這種有意義的句子。圖 11-15 即依此繪製。讀法：曲線愈靠左上愈好（同位元率下失真更小，或同失真下位元更少）。

21. 軟門檻為什麼要把留下來的大係數也往零收縮一個門檻值?這樣做的好處與代價各是什麼?為什麼軟門檻的最佳門檻通常比硬門檻低?

    提示與參考解答：觀測到的係數是 w = s + n（訊號加雜訊）。硬門檻在 |w| > T 時原封不動保留 w，等於把 s + n 當成 s，雜訊整個留了下來;軟門檻的 −T 就是用來抵消這一項，平均而言可以去掉一部分附著在訊號上的雜訊。代價是偏差：一個 w = 300 的強邊緣在 T = 75 時會變成 225，邊緣因此變淡。所以軟門檻是拿偏差換變異數。既然它本來就會額外削掉一截，門檻再設高就削過頭，最佳門檻自然較低。圖 11-17 實測：軟門檻在 1.50σ 達到 27.10 dB，硬門檻在 3.25σ 達到 26.54 dB。詳見 11.6.4 節的推導。

22. VisuShrink 建議的門檻是 T = σ·√(2·ln N)，其中 N 是係數個數。對一張 512×512 的影像，這個門檻相當於幾倍的 σ?實測顯示它的去雜訊效果明顯低於最佳門檻，這是否代表 VisuShrink 是錯的?

    提示與參考解答：N = 512×512 = 262,144，ln N ≈ 12.48，√(2 × 12.48) ≈ 5.0，所以 T ≈ 5.0σ。圖 11-17 實測這個門檻只得到 24.32 dB，比最佳的 27.10 dB 低了

2.78 dB。但這不代表它錯，而是它在最佳化另一個目標：VisuShrink 的設計準則是「以高機率讓純雜訊的係數全部被歸零」（N 個高斯變數的最大值約為 σ√(2 ln N)），追求的是視覺上的乾淨，不是 PSNR 最大。代價是連弱的真實細節也一併抹掉。這是工程上很常見的一課：先確認指標衡量的是不是你真正要的東西，再去談哪個方法比較好。

23. 在多焦點影像融合中，「直接平均」「逐點取最大」「小波逐子帶取細節較強者」三種策略的成效如何?為什麼前兩者會失敗?

提示與參考解答：圖 11-18 以人工製造的來源（同一張清晰影像左右各模糊一半，故有正確答案可比對）實測：直接平均 29.94 dB、逐點取最大 27.21 dB、小波融合 37.86 dB。直接平均失敗是因為它無差別地把清楚與模糊的部分各取一半，兩邊都變半糊;逐點取最大更差，因為「比較亮」與「比較清楚」是兩回事，它會把亮點誤判成細節，還可能破壞原本的明暗關係。小波融合的做法是在每個子帶的每個位置比較兩張影像誰的細節係數較強就選誰，等於在「頻率 × 位置」的網格上逐格做選擇——這正是小波同時握有兩種資訊才做得到的事。驗證：融合後左右兩半的拉普拉斯變異數都回到了原影像的水準。

24. 課本常說浮水印要嵌在中頻，理由是「低頻太明顯、高頻不耐壓縮」。若把三個位置的嵌入強度調到隱蔽性完全相同再比較強健度，結果會是什麼?由此說明浮水印設計真正的取捨是什麼。

提示與參考解答：圖 11-19 用二分搜尋讓三者的 PSNR 都等於 40 dB 再比較，結果低頻在六種攻擊下全部勝出：JPEG q20 之後低頻仍有 0.945，中頻 0.283，高頻只剩 0.020;連 SSIM 也站在低頻這邊（0.9912 對 0.9457、0.9454）。原因是清單上每一種攻擊（壓縮、模糊、加雜訊、縮放）本質上都是低通運算，它們摧毀的正是高頻。那為什麼不全塞低頻?因為容量：LL3 只有 4,356 個係數，第 1 層有 66,049 個，差 15.2 倍。所以真正的取捨是「強健度對容量」，而不是「強健度對隱蔽性」——藏一個序號就往低頻，藏一整段訊息才需要往高層走。補充：PSNR 與 SSIM 都低估低頻失真的可見度（平坦區會出現淡暈影），實務上仍會把低頻的嵌入強度再壓低一些。

25. (綜合題)要量化「方塊效應」有多嚴重，可以計算影像在 8 的倍數欄位上的平均梯度，再除以其他欄位的平均梯度。為什麼要做這個除法?若直接用「8 的倍數欄位的平均梯度」當指標會有什麼問題?

    提示與參考解答：直接用絕對值會被影像內容干擾——一張本來就布滿細節的影像，所有欄位的梯度都大，8 的倍數欄位自然也大，但那不是方塊效應。除以其他欄位的平均梯度等於做了正規化，衡量的是「格線位置的不連續程度是否異常高於一般位置」，與影像整體的紋理強弱無關。判讀時要看它與原圖基準的落差：圖 11-20 中原圖的指標是 1.03（格線位置並無特殊之處），JPEG 壓到 0.521 bpp 後升到 1.89，而小波在更低的 0.504 bpp 下仍是 1.03。這個設計思路值得記住：好的指標要能把「想測的效應」與「資料本身的性質」分開，否則測到的只是資料長什麼樣子。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 6-7、8(小波、多解析度與影像壓縮)。
- Bradley, J. N., Brislawn, C. M., & Hopper, T., The FBI Wavelet/Scalar Quantization Standard for Gray-Scale Fingerprint Image Compression, 1993.(本章叫獸開講之史料來源)
- Daubechies, I., Ten Lectures on Wavelets, SIAM, 1992.(小波理論經典)
- Mallat, S., A Theory for Multiresolution Signal Decomposition: The Wavelet Representation, IEEE TPAMI, 1989.
- Wallace, G. K., The JPEG Still Picture Compression Standard, Comm. ACM, 1991.
- Taubman, D., & Marcellin, M., JPEG2000: Image Compression Fundamentals, Standards and Practice, Springer, 2002.
- 補充教材與課程資訊：@prof(https://page.line.me/prof)